A plan for proving Ochr consistent and type safe

The route to be followed, the lemmas it needs, and where it could fail

Draft for discussion · 9 November 2026 · a plan for a proof, not a proof

Summary. The claim this plan serves is that a language like Ochr makes a good general-purpose language. That needs two guarantees, *consistency*, so that its proofs can be trusted, and *type safety*, so that accepted programs never crash or go wrong when run, and Ochr promises a third, `Id` *adequacy*: programs it proves equal can be swapped in any context. The plan proves the first two by *meaning preservation*, and the third from the same meanings with a context lemma. Types are given meanings in a set model, in which `False` denotes the empty set. Every state the checker reaches while checking a definition is given a meaning too, as a function of what its unknowns stand for: an *assignment* gives each unknown any element of its type's meaning, non-computable functions included. The central theorem has three parts, proved together by one ordinary induction: each step of the checker keeps the meaning of its state under every assignment, or loses only information; every value the checker stores lies in the meaning of its type, whatever the borrows still live may later write; and the state stays well formed. An accepted definition therefore means an element of its type, and a closed proof of `False` would be an element of the empty set. All four planned paper tests have passed (Section 6): the set model exists on a slice chosen to contain everything that made its definition doubtful; states with live borrows have a meaning, as functions of their borrows' final contents, and the step lemmas hold on a higher-order slice, much of both checked in Lean; each of the four closed proofs of `False` the checker once accepted and has since fixed is a step at which one of the theorem's three parts fails; and an audit of every decision the checker makes left one item to settle. The one failure found is a fifth closed proof of `False`, conversion identifying two values that hold a moved-out place, which fails the monotonicity the theorem requires of computing steps; it is fixed on main by D83. Type safety reuses the same meanings: a closed run of an accepted function means what the reference run of its call says, and that reference run never goes wrong; test 3 checked it on a slice, with no checker bug found: errors at runtime reflect to the reference run or, at one-element types, to the checker's derivation; `J` at runtime is its transported term; and erasure holds by construction. What remains for it is chiefly a realizability relation for calls through function values (Section 4.3.6). The rest is a short list of residual lemmas (Section 8).

Contents

1 What this document is . . . 3
2 The calculus, as far as the plan needs it . . . 3
2.1 Terms, values and neutrals . . . 3
2.2 States, borrows and resolution . . . 4
2.3 The machine, closing off and sealed programs . . . 4
2.4 Observation, footprints, `Id` and `Eq` . . . 5
2.5 Erasure and private copies . . . 5
2.6 Typing . . . 5
2.7 Conversion . . . 7
2.8 Refinements and the two paths . . . 7
3 The target . . . 7
4 The shape of the argument . . . 9
4.1 The set model . . . 9
4.2 Conversion is sound . . . 14
4.3 Meaning preservation . . . 15

5 The principles the checker enforces 23
5.1 Locality 23
5.2 Closing off 23
5.3 Worlds 24
5.4 Wholeness 24
5.5 Erasure and captures 25
5.6 Recursion 25
5.7 Observation 26
6 Tests before mechanisation 26
6.1 Test 0: the model exists, on a slice 26
6.2 Test 1: the known proofs of `False` are failed steps 27
6.3 The census of the checker's decisions 28
6.4 Test 2: a higher-order slice 29
6.5 Test 3: type safety on the higher-order slice 30
6.6 Audit: what a context can observe 31
7 The lemmas and their status 31
8 The order of work 33
9 Where the plan could fail 33
10 Open design questions 35
11 An earlier version 35

1 What this document is

This is the plan we intend to follow to prove that Ochr is consistent and type safe, and that programs it proves equal can be swapped: the shape of the argument, the definitions it needs, the lemmas it reduces to, which of those are already proved and where, and where the plan could fail. It is written as a section of theory work would be, so the statements are precise where they can be, and where they cannot be yet, that is said.

The object of study. The calculus is the one implemented by the Lean checker in `ochr/core/lean/Ochr/` (chiefly `Syntax.lean`, `Basic.lean`, `Env.lean`, `Machine.lean`, `Decide.lean`, `Obs.lean`, `Check.lean` and `Run.lean`) and laid out rule by rule in the appendix of the Ochr paper. Section 2 restates every part of it that the plan relies on, so that the plan can be followed without either.

Status labels. Each lemma carries one of four labels.

- **proved on the heap slice**: proved, without `sorry`, in Lean for the *heap slice* (`ochr/core/lean/Ochr/Heap/`), a second evaluator for part of the calculus, in a heap-based style. Not a statement about the checker.
- **reduced to X**: shown to follow from the named lemmas X by an argument given here.
- **paper argument**: an argument exists at the level of detail of this document, not checked in Lean.
- **open**: no argument yet.

Design questions that must be decided before an argument can be written are marked **open design question**.

Route. The plan follows *meaning preservation*. Every type, value and checker state is given a mathematical meaning (a *denotational model*), and each step the checker takes is shown to keep the meaning of its state. Truth is defined in the model, so a step that keeps meaning cannot lead the checker from true premises to a false conclusion. This is the usual soundness argument for *symbolic execution* (Section 4.3), with set-theoretic meanings in place of concrete machine states. No second, concrete run is involved: the checker's run is compared with what it means. Another route, translating each accepted program into Lean and having Lean's kernel check it, gives consistency one program at a time; it is not part of this plan, though its results are used in Section 9 as evidence.

2 The calculus, as far as the plan needs it

2.1 Terms, values and neutrals

Terms and types share one grammar: variables, the universes `Prop` and Type_i, n-ary Π-types, recursive functions **fix** $f\left(\overline{x} : \overline{A}\right) : B$ **by** $x_j \coloneqq t$ (a `fix` without `by` is a λ), saturated calls, inductive types $\mathsf{D}(\overline{a})$ and constructors $\mathsf{C}(\overline{t})$, the primitive equality `Eq` $A\ t\ u$, transport $\mathsf{J}(A, a, b, P, h, t)$ and **rewrite** h **in** t, borrow types $\&A$, places $p ::= x \mid *p \mid p.g$, borrowing $\&p$, assignment $p \coloneqq t$, `let`, sequencing $t; u$, **match** p $\{\mathsf{C}_1(\overline{y}_1) \Rightarrow t_1, ...\}$ with any number of arms, **split** f **in** t, and `Id` $A\ t\ u$, the equality of two computations.

Every type of data and every connective is an inductive declaration **inductive** $\mathsf{D}\ \left(\overline{x} : \overline{X}\right) : s \coloneqq \mathsf{C}_1\left(\overline{y}_1 : \overline{B}_1\right) \mid ...$ with $s \in \{\mathsf{Prop}, \mathsf{Type}_0\}$. The library declares `Nat`, `Unit`, `Pair`, `False` (no constructors), `True` (one constructor `I`, written `refl`) and `And`. A field's type may mention the type being declared, other declared types, parameters, earlier fields of the same constructor (the fields form a *telescope*), calls of earlier *type functions* (definitions whose codomain is a sort), and equations between *first-order data* (types built from inductive types and parameters, with no Π, no `&` and no sort); never `Π` or `&`. Parameters whose declared type is syntactically a sort are *type parameters*; the others are *value parameters*, and a recursive occurrence of `D` in a field may take any arguments at them (*non-uniform* occurrences). A family indexed by values is written by fixing the index with an equation field (*Fording*): `Vec(A, n) := VNil(h : Eq(Nat, n, Z)) | VCons(m : Nat, x : A, xs : Vec(A, m), h : Eq(Nat, n, S(m)))`.

$\&A$ requires $A : \mathsf{Type}_0$, and `&` occurs only as the whole declared type of a parameter, result or annotated term: never inside another type and never as the result of computing a type. So there are no borrows inside data and no borrows of borrows. Universes are not cumulative, and `Prop : Type₁` beside `Type₀ : Type₁`.

Values are constructor values $\mathsf{C}(\overline{a};\overline{v})$ (recording the inductive's parameters $\overline{a}$), the proof value $\star$, top-level function names, closures $\langle\overline{\kappa}\vdash \mathbf{fix}\ ...\rangle$ (code with captured values $\overline{\kappa}$), Π-closures $\langle\overline{\kappa}\vdash \Pi\big(\overline{x}:\overline{A}\big).B\rangle$ (types are values), type values, borrows $\mathrm{borrow}_\ell v$, loans loan_ℓ, the moved-out marker $\bot$, and *neutrals*. A neutral is an *abstract value* σ (an unknown of a known type $\Delta(\sigma)$, introduced only by the checker) or a *sealed program* $\lceil t\rceil$ (a program with no free variables whose run is stuck). A value is *borrow-free* if it contains no borrow and no live loan. A value is *closed* if it contains no free variables, no abstract values, no loans and no $\bot$: nothing whose meaning depends on an environment, on what the unknowns stand for, or on contents held elsewhere in the state. A closed value therefore means the same thing wherever it appears, which is what lets the model give it a meaning $[\![v]\!]$ on its own (Section 4.1). A sealed program has no free variables but is stuck on some abstract value, so it is not closed; nor is anything containing one.

2.2 States, borrows and resolution

An environment Ω is a stack of frames, each binding variables (with declared types) to values, plus temporaries. A *state* adds Δ (the types of abstract values), ρ (the refinements and generalisation records made so far), the goal while a definition is checked, and the recursion stack. $\mathrm{content}_\Omega(p)$ follows a place through variables, borrows ($*\,p$ looks inside $\mathrm{borrow}_\ell v$ at v) and fields.

Borrows. $\&p$ moves the content v of p into $\mathrm{borrow}_\ell v$ and leaves loan_ℓ at p. *Ending* the borrow ([End]) replaces it by $\bot$ and substitutes its content for every occurrence of loan_ℓ: a loan is a variable bound by its borrow. Every access to a place is preceded by [Access], which ends every borrow whose loan lies on the path to the place or inside its content (for a match, which inspects only the head: on the path, at the head, or anywhere inside a neutral head). Reading non-copy data moves it out, leaving $\bot$; data of a *copy type* (`Unit`, sorts, propositions, inductives declared `copy`, and non-recursive inductives whose fields are copy types) is copied, copy-ness being read from the place's declared type, in which a type parameter is never a copy type; and $\mathbf{clone}(p)$ copies explicitly. The machine's rules for these steps are [Borrow], [Read] (moving data out), [Move] (moving a borrow), [Copy], [Clone] and [Assign]; [Ctor] builds a constructor value. A borrow must be whole (no $\bot$ inside) when it ends, is returned, or is passed to a call that closes off.

Dropping. When an owned value dies holding a live loan, [Drop] ends the borrower if the borrower is held in a binding, and fails if the borrower is a value in flight (a temporary such as a result being returned). So a local may die while a variable that borrowed it is never used again, but nothing may return a borrow of its own local.

Owners and resolution. $\mathrm{owners}_\Omega(\ell)$ follows each occurrence of loan_ℓ outward: through the content of an enclosing borrow_m to $\mathrm{owners}_\Omega(m)$, otherwise to the owned position holding it. The *resolution* $\rho(\Omega)$ ends every borrow; $\rho_\Omega(w)$ is w with each live loan replaced by the resolved content of its borrow.

Well-formedness. A state is well formed when (WF1) each label has at most one borrow, which is the whole value of a binding or temporary, never inside data, a sealed program, a closure or a type; (WF2) every loan's borrow is in Ω, except a loan inside a sealed program whose borrow is outside the run (an *inert* loan); outside sealed programs a label occurs as a loan at most once, inside them any number of times; (WF3) the relation "a loan of ℓ occurs in the content of borrow m" is acyclic; (WF4) closures and Π-closures capture no borrow and no $\bot$, and sealed programs are closed; (WF5) at every call point each argument is loan-free or is $\mathrm{borrow}_\ell u$ with u loan-free; (WF6) each binding's value is $\bot$, or a loan whose borrow's content has the binding's type, or a value of the binding's type, and every σ has a type in Δ.

2.3 The machine, closing off and sealed programs

The machine is big-step and deterministic: $\langle\Omega,t\rangle \Downarrow \langle\Omega',v\rangle$, or the run is *stuck* (it must inspect a neutral), or it fails. Calls evaluate their arguments left to right into temporaries, push a frame binding the parameters, run the body, and pop the frame, dropping its bindings. Types and proofs are *erased*: an erased term runs on a private copy of the state, which is discarded (Section 2.5).

Closing off ([Close]). When a call $f(\overline{w})$ is stuck, the call itself returns. Let the borrow arguments be $w_i = \mathrm{borrow}_{\ell_i} u_i$ ($i \in I$) with each u_i loan-free, let L be let $c_i = u_i$ for $i \in I$, and C the call $f(\overline{a})$ with $a_i = \&c_i$ for $i \in I$ and $a_i = w_i$ otherwise. If f's declared codomain is borrow-free, the call returns $\lceil L; C\rceil$ and each loan_{ℓ_i} is filled with $\lceil L; C; c_i\rceil$. If it is $\&T$, the call returns $\mathrm{borrow}_k\lceil L;\ \mathrm{let}\ r = C;\ *r\rceil$ for a fresh k, and each loan_{ℓ_i} is filled with

$\lceil L; \text{let } r = C; * r := \text{loan}_k; c_i \rceil$: the fills have a *hole* loan_k for the final content the caller will write through the returned borrow, which the caller's eventual [End] of borrow_k supplies by substitution. A result whose declared type at the call reads as `Unit` is `()`, and so is the fill of a lent place whose declared type is `Unit`. A call whose head is a neutral is stuck at once and closed off the same way.

Sealed programs. Values are kept in normal form. A sealed program's normal form is its run from the empty environment, except that its head call C may not itself close off ([Seal]'s head guard); so a sealed program either runs to a value or stays exactly as it is. A loan whose borrow lives outside the run is inert: the run treats it like an abstract value.

Stuck blocks. A stuck `match` that is not the body of a call is closed off as a call of an anonymous function of the places it uses. Pattern variables are first resolved to the places they denote; the block takes each maximal place by move if some arm moves it, by borrow if some arm writes or borrows it, and by copy otherwise. A stuck `J` is closed off the same way, with its transported term as the block's one arm.

2.4 Observation, footprints, `Id` and `Eq`

The *footprint* $W_\Omega(t, u)$ is the set of owners of the roots of the places that t or u borrows or assigns, and of the borrow variables they read whole, in a canonical order read off the syntax. A place that is only read contributes nothing. The owner of a variable holding a borrow is the owners of its loan; of an owned variable, the variable itself.

The *observation* $\text{Obs}^W_\Omega(t)$ runs t on a private copy of Ω, ends every remaining borrow, and returns the tuple of the result and the contents of the positions of W. A borrow-typed result is observed after writing a given fresh σ_w through it, so that its owners show where it points ([Obs-borrow]); only conversion observes such results.

$\mathsf{Id}(A, t, u)$ computes, at Ω, to $\text{and}(\text{eq}(A, r, r'), \text{eq}(T_1, w_1, w_1'), ...)$, where $\text{Obs}^W_\Omega(t) = (r, \overline{w})$, $\text{Obs}^W_\Omega(u) = (r', \overline{w'})$ and $W = W_\Omega(t, u)$; A must be borrow-free. `Eq` computes through the smart constructor `eq`: on two values of one constructor it is the conjunction of the fields' equations (injectivity; for a *dependent* constructor, one some of whose fields' types mention earlier *index* fields, only when the index fields are convertible), on values of different constructors of one inductive it is `False` (disjointness), on convertible values (equal by definitional equality, Section 2.7) it is `True`, and otherwise, or whenever $\bot$ occurs in a side, it is the stuck type $\mathsf{Eq}\ T\ v\ w$. `and` drops `True` conjuncts. $\mathsf{J}(A, a, b, P, h, t)$ computes to t only when $a \equiv b$; otherwise it is stuck and closed off.

2.5 Erasure and private copies

A term's *declared type* is read from the declared types of its heads without normalising (a variable's declared type, a call's callee codomain term, a constructor's inductive, an annotation, a tail's or arms'). A term's *declared sort* is the sort of its declared type, read the same way. A term is *erased* when it is a proof (its declared type has declared sort `Prop`) or forms a type (a type former, a call of a function returning a sort, or anything in a *type position*: an annotation, a Π domain or codomain, a type slot of `Eq`, `Id` or `J`, or a parameter or field type). An erased term is a *statement*. A run is in *erased mode* while it runs a statement, and in *runtime mode* while it runs code. A Π-type's *class* (returns types, returns proofs, other) and whether it returns a borrow are read from its codomain term when it is formed, and Π-types that differ in either are not convertible. *Sorts are syntactic*: whatever is written in a type position must have a declared type that is syntactically a sort, so a type's declared sort is its computed sort. Every erased term runs on its own private copy of the state, which is discarded. In the checker these decisions are functions of syntax, declarations and declared types only, collected in one module (`Decide.lean`) that cannot run the machine.

2.6 Typing

The typing judgement $\Omega \vdash t \Downarrow v : A \dashv \Omega'$ is the machine run with types: each rule performs the machine's step and computes a type. Where the machine would be stuck on the program being checked, typing splits instead. The rules the plan relies on:

- **[Call-type].** The type of a call is its callee's codomain evaluated, on a private copy, with the parameters bound to the arguments' values; each argument's type must convert to the corresponding parameter type, evaluated the same way.

- **[T-Call].** The call's type is its [Call-type]; then the callee runs in the machine, where a stuck body closes off. For a proof-typed call ([T-Call-proof]) the body is never run.
- **[Rec].** In the body of **fix** f... **by** x_j, f occurs only as the head of a call (also inside nested functions and block arms), and every call of f, recognised by the name f binds for itself or a nested function's capture of that name, passes in position j a strict subterm (through any field, and through the borrow for &D) of the *entry value* of x_j (its value when the call started), as refined so far.
- **[Split].** A match on a place whose content is an abstract value σ checks each arm i under the refinement $\sigma := \mathsf{C}_i(\overline{\sigma}_i)$ with fresh field values typed by the constructor's telescope; the refinement is applied to the environment, every stored type, Δ and the goal, with sealed programs re-normalised. Then the match is closed off as a stuck block, and checking continues once from the unrefined state.
- **[Split-gen].** A match on a sealed program n first *generalises* it: replace n everywhere by a fresh σ of the place's type, record $n := \sigma$ (so a later derivation of n normalises to σ), then split σ. **split** f **in** t does the same for the first stuck call of f in the goal.
- **Worlds.** Each arm of a split is checked in its own *world*. Leaving an arm discards everything it established except fresh-name counters and the types of abstract values it created. A generalisation record, and an η-*refinement* (refining σ to $\mathsf{C}(\overline{\sigma})$ because σ's type has only the one constructor C), lives until the arm that made it ends: the constructor follows from the type alone, but the fields' types are the arm's.
- **[Def].** A definition is checked at its *generic call*: each borrow parameter is bound to $\mathrm{borrow}_{\ell_i}\sigma_i$ with a fresh owned cell $c_i \mapsto \mathrm{loan}_{\ell_i}$ in a bottom frame, each proof parameter to $\star$, each other parameter to a fresh σ_i. The goal G is the [Call-type] of that call. The body is checked in *tail position* (where only trailing drops follow: the body itself, and the tails of `let`s, sequences and arms in tail position), producing a finite *case tree* whose leaves (Ω', v, A) are the states, values and types at the end of each path; on every leaf the frame's pop must succeed and $A \equiv G'$, the goal refined along that path. The tail rules are [Tail-let] and [Tail-seq] (run a `let` or sequence, then continue), [Tail-match] (a match on a known constructor), [Tail-split] (a split on an abstract value, one subtree per arm), [Tail-gen] (generalise a sealed program, then split), [Tail-prop] (a match on a proof, one subtree per arm) and [Tail-end] (a leaf). Constants are checked from the empty state ([Const]); inductive declarations by [Ind-decl], which checks the conditions of Section 4.1.3.
- **Matches on proofs.** A match whose arms name constructors of an inductive in `Prop` is decided by the scrutinee's type, never its content ($\star$). It may produce data or a type only if the inductive is a *subsingleton* (no constructors, or one whose fields are all proofs). `match h {}` on h : `False` is ex falso; in tail position it contributes no leaves.

> **Quick explainer: large elimination and subsingletons.** *Eliminating* a value means taking it apart with a match. Matching on data is unrestricted. Matching on a proof to produce data or a type is called *large elimination*, and it needs care, because proofs are erased and every two proofs of one proposition count as equal (*proof irrelevance*). If the result could depend on which constructor the proof used, two "equal" proofs would give different results. For example, if $\mathsf{Or}(P, Q)$ could be matched to produce a `Bool`, the proofs $\mathsf{Inl}(t)$ and $\mathsf{Inr}(t)$ of `Or(True, True)` are equal yet would give `true` and `false`, so `true = false`. Large elimination is therefore allowed only from a *subsingleton*, a proposition with at most one possible proof: no constructors (`False`, where there is nothing to match) or one constructor whose fields are all proofs (`True`, `And`, an equation). Lean has the same rule.

Dependent fields. A field whose type mentions earlier fields makes its constructor *dependent*; the earlier fields it mentions are *index fields*. Writing, borrowing or moving a field of a dependent value *opens* it ([Open]): while open, a data field is typed by its content, assigning it is a strong update, and a write or borrow through an index field sets the proof fields whose type mentions it to $\bot$ (an error to read until reassigned). A value must be *whole* again (every dependent constructor inside it at its telescope type, no $\bot$ inside) wherever it passes to code that relies on its type ([Repack]); a part still lent at that point is checked as an unknown of its borrow's type, since the borrow may still write anything of that type there. Where a checked body returns, its result and every place it holds by borrow must be whole ([Return]); [Def] and closures check this at their body's return, and a stuck block at the end of each arm (it is an anonymous function, so its arms are its bodies).

2.7 Conversion

Conversion $v \equiv w$ is the *least* relation closed under: [Conv-refl] identity up to bound-variable names; [Conv-cong] same outermost former with pairwise convertible components (a constructor value's recorded parameters ignored; sealed programs compared by their program text); [Conv-unit] $\mathsf{And}(\mathsf{True}, P) \equiv P \equiv \mathsf{And}(P, \mathsf{True})$; [Conv-pi] two Π-closures with the same class and borrow flag whose parameter types and codomains, evaluated at the generic arguments of the first, are pairwise convertible; [Conv-fun] two function values whose Π-types are convertible, whose captured values are pairwise convertible, and whose generic calls have convertible observations (with one shared σ_w for a borrow result). There is no η-rule; values are `()` wherever their declared type reads as `Unit`, and every proof is $\star$.

2.8 Refinements and the two paths

Definition 1 (refinement). A *refinement* α assigns values to abstract values, $\sigma \mapsto v$. It *respects* a state Ω when each σ it refines is an abstract value of Ω and $\alpha(\sigma)$ is borrow-free and has a type convertible with $\Delta(\sigma)$. It is applied to a state, value or term by substituting and then restoring normal form: every sealed program that changed is re-run ([Seal]) and every `Eq` type rebuilt by `eq`. We write $\Omega\alpha$, $v\alpha$, $t\alpha$; α *covers* Ω when it assigns a value to every abstract value of Ω. More generally, $\Omega[w/a]^+$ replaces every occurrence of an abstract value, loan or sealed program a by w in the values of Ω, its stored types, Δ and the goal, and then restores normal form; $\mathrm{nf}(v)$ is the normal form of v.

Refinements cover the three ways the checker instantiates an unknown: a case split ($\sigma := \mathsf{C}(\overline{\sigma}')$), a call site (a parameter's generic σ instantiated with the argument), and an abstract function instantiated with a closure. Borrow-freeness costs nothing: a borrow parameter's generic value is $\mathsf{borrow}_\ell\sigma$ with σ borrow-free, and since `&` is never produced by computation and never an argument, no unknown type is ever refined to a borrow type.

The two paths. Every statement is evaluated along two paths: at a definition's generic call, where calls and matches on abstract values close off, and directly at each instance, where they run. [Call-type] and [Split] identify the results, so a proof checked on the first path is used on the second. If the paths take any decision differently, and the decision changes a result, a statement proved on one path can be false on the other: a closed proof of `False`. The checker's design principle is that *no decision that changes a result is read from a normal form.* This plan needs less: a decision may read anything, provided its outcomes mean the same, are chosen by syntax the model reads the same way, or are ordered by information (Section 4.3.4).

The plan does not compare the two paths. It reads the first path's states under *assignments* (Definition 10), which give each unknown a mathematical element of its type's meaning rather than a machine value.

3 The target

Definition 2 (accepted program). A *program* is a finite list of inductive declarations and definitions, read in order. It is *accepted* when each item is accepted by [Ind-decl], [Def] or [Const] against the signature of the items before it, the library's declarations (`Pair`, `False`, `True`, `And`, with `Nat` and `Unit` built in) coming first. It is *hole-free* when no term in it is a hole (`?`, also written `sorry`). Every definition has a body, so holes are the only axioms.

Target A (consistency of the calculus). Assume ZFC with one inaccessible cardinal per universe level used (both defined in Section 4.1.1). For every accepted, hole-free program, no definition without parameters has a type that evaluates to `False`. Equivalently, no closed term has type `False`, nor any type, such as `Eq(Nat, Z, S(Z))`, that computes to it.

Target B (type safety). Let $\mathcal{P}$ be an accepted, hole-free program, d an accepted *data function* of $\mathcal{P}$ (its codomain is data or a borrow type), and $d(\overline{v})$ a *closed call*: every argument is a closed value *built by accepted code* (a constructor value, or a closure or global of $\mathcal{P}$, holding no unknown and no sealed program), every borrow argument is a borrow of a cell holding such a value, and every proof parameter's type, evaluated at the earlier arguments, is *true* in the set model. Then the *erased run* of the call (the erased program's run: the machine in runtime mode, with statements not run, every proof $\star$, and $\mathsf{J}(A, a, b, P, h, t)$ evaluating to t unconditionally,

D85) terminates; raises no error and never gets stuck; returns, if d returns a borrow, a live borrow whose content is whole; and leaves a result and final contents of the borrowed cells that mean what the reference run of the call says (Definition 13).

Target C (`Id` adequacy). If an accepted, hole-free lemma proves $\mathsf{Id}(A, t, u)$, then at every instance where its hypotheses hold, t and u are *contextually equivalent*: at every accepted program position holding t, replacing t by u gives a run with the same result, the same final state and the same errors.

Why three. The plan serves the claim that a language like Ochr makes a good general-purpose language. Its proofs must be trustworthy, which is Target A, and its programs must not crash or go wrong when run, which is Target B: what any verified language owes its users. Target C is Ochr's distinctive promise: programs proved equal can be swapped, the substitution principle behind D84 (drop at the join). Neither of the first two implies it (Section 4.3.7). The statement of Target C is a draft, to be sharpened, and no Lean statement of Target B or Target C is fixed yet.

Why "built by accepted code". A closure from outside the program could loop or go wrong; arguments built by the program's accepted code behave as their meanings say (Section 4.3.6, termination). **Why "true", not "proved".** It is the stronger statement: it covers a caller that obtains its hypotheses outside Ochr. It is also why `J` at runtime must not decide by conversion.

Termination. Ochr stays total for now: every recursion passes [Rec], so type safety includes termination. General recursion is desirable, but is deferred until a pure, consistent and sound core exists.

The Lean statement of Target A against the checker is `Ochr.Consistency.consistency`:

```
theorem consistency (q : Program) (fuel : Nat)
    (holes : NoHoles q) (acc : Accepted (Prelude.decls ++ q) fuel) :
    ∀ d ∈ q, ¬ d.IsFalseConst
```

`Prelude.decls` is the library's declarations. `Accepted p fuel` says that the checker's entry point `checkProgram` gives every declaration of `p` the verdict *accepted* at that fuel (not *rejected*, and not *undecided*, which is what running out of fuel or a depth limit gives). `IsFalseConst` says a declaration is a definition with no parameters whose declared type is the identifier `False`.

How the two statements relate.

- **Rules and checker.** Target A is about the rules; the Lean statement is about the checker that implements them. Every step of this plan is phrased so that it can be stated against the checker's functions, so a completed plan would prove both, the Lean form being the stronger since it also excludes implementation errors.
- **A literal `False` loses nothing.** If an accepted program had a parameterless c whose type merely computes to an empty type, one more fresh declaration turns it into a literal one: `def B : False := c` when the type computes to `False`; a one-arm match for `False ∧ ⊤`; an annotated zero-arm match for a user's empty inductive. The Lean statement quantifies over all programs, so it covers these.
- **Fuel.** Exhaustion is an outcome of its own that no rule below the verdict catches, and a caught failure cannot refund fuel, so a decided verdict is the same at every larger fuel. The Lean statement quantifies over every fuel and does not need this fact, but a proof may use it to fix a fuel large enough for a given run.
- **Totality.** Every function the checker runs is total, with explicit fuel where recursion is not structural, so the kernel can unfold every step. The one exception, an opaque address used to key the editor's location table, is never consulted when locations are off, which `Accepted` ensures.
- **Universes.** Lean's strength is ZFC with n inaccessibles for every $n < \omega$. If the set model for Ochr needs one inaccessible per level at once, the uniform Lean statement is unprovable in Lean (Gödel's second incompleteness theorem: a theory cannot prove the consistency of one at least as strong, Section 4.1.1), and must be weakened to bounded universe levels, made relative to an assumed model, or stated per program (Section 10, Q1).

> **Quick explainer: Lean's kernel and its axioms.** Lean's *kernel* is the small trusted core that checks every proof; tactics and the elaborator only produce proof terms for it to check, so a Lean theorem is believed exactly as far as the

kernel is. Beyond its core type theory, Lean has three standard axioms: `propext` (two propositions that imply each other are equal), `Quot.sound` (elements related by a relation are equal in its quotient) and `Classical.choice` (from a proof that something exists, pick one). All three hold in Lean's set model. `#print axioms t` lists the axioms a theorem t depends on; an unfinished proof shows up there as `sorryAx`. A `partial def` is opaque to the kernel: it can run, but a proof cannot unfold it, which is why a provable statement about the checker needs every function the checker runs to be total.

4 The shape of the argument

The argument has three parts: a set model that says what "true" means (Section 4.1); a meaning for every state the checker reaches, with a theorem that the checker's steps keep it (Section 4.3); and consistency as a corollary (Corollary 16). Type safety follows from the same theorem and four further lemmas (Section 4.3.6), and `Id` adequacy from it and a context lemma (Section 4.3.7).

4.1 The set model

4.1.1 Background: sets, universes and models

This subsection defines everything the model uses. It follows the standard set model of Lean (Carneiro, *The Type Theory of Lean*, a copy of which is at `docs/papers/theory-of-lean.pdf`), but nothing below depends on having read it. For readers who want the full construction for Lean, of which Section 4.1 is an adaptation, the relevant parts are, optionally: §1.2 (why sets model a type theory, the two-element `Prop`, and why inaccessible cardinals are needed); §6.2 (the interpretation map, with universes $U_{n+1} = V_{\kappa_n}$ and the clauses for Π, `∀` and `=`); §6.2.1 (the least-fixed-point construction of inductive types in ZFC); and §6.3, Theorem 6.7 (the soundness of the model: types denote sets, proofs denote $\bullet$, convertible terms denote the same thing).

ZFC, and consistency relative to it. ZFC is Zermelo–Fraenkel set theory with the axiom of choice, the usual foundation of mathematics. A theory T is *consistent* if it does not prove a contradiction. By Gödel's second incompleteness theorem, a consistent theory cannot prove its own consistency, nor that of a theory at least as strong; so a consistency proof for a strong type theory is always *relative*: "if ZFC plus some axioms is consistent, so is the type theory".

Quick explainer: countable and uncountable sizes. Two sets have the *same size* if there is a bijection between them (a pairing that matches every element of one with exactly one element of the other); $|A|$ is the size of A, and $|A| \le |B|$ means A can be matched one-to-one into part of B. A set is *countable* if it is finite or has the same size as the natural numbers $\mathbb{N}$, that is, its elements can be listed $a_0, a_1, a_2, \ldots$; the integers, the rationals and the set of all finite strings are countable. A set is *uncountable* if it is not. Cantor's theorem says that no set A has the same size as $\mathcal{P}(A)$, the set of all its subsets: $\mathcal{P}(A)$ is always strictly bigger. So $\mathcal{P}(\mathbb{N})$, equivalently the real numbers or the set of all functions $\mathbb{N} \to \mathbb{N}$, is uncountable, and the sizes go on growing forever: $\mathbb{N}$, $\mathcal{P}(\mathbb{N})$, $\mathcal{P}(\mathcal{P}(\mathbb{N}))$, …

Quick explainer: cardinals. A *cardinal* is a size. The size of $\mathbb{N}$ is written $\aleph_0$ ("aleph-nought"), the smallest infinite cardinal; the cardinals bigger than it are the *uncountable* cardinals. If $\lambda = |A|$, then 2^λ is the size of $\mathcal{P}(A)$ (a subset of A is a choice of in or out for each element, hence the notation); by Cantor, $2^\lambda > \lambda$ always. Cardinals are ordered by $\le$ above, and "fewer than κ sets" means a collection of them whose size is less than κ.

Quick explainer: ordinals. Ordinals extend counting past the natural numbers. They measure *position* in a sequence, not size: $0, 1, 2, \ldots, \omega, \omega+1, \omega+2, \ldots, \omega\cdot 2, \ldots$, where ω is the first position after every natural number. Every ordinal other than 0 is either a *successor*, which has an immediate predecessor ($3 = 2+1$; $\omega+1$ comes straight after ω), or a *limit*, which has none and is reached only as the limit of all smaller ordinals (ω, $\omega\cdot 2$, …). Different ordinals can have the same size: ω, $\omega+1$ and $\omega\cdot 2$ are different positions but all countable. Each cardinal is identified with the first ordinal of that size, so $\aleph_0$ is ω, and every infinite cardinal is a limit ordinal, though most limit ordinals are not cardinals. This identification is what lets a cardinal κ index a stage V_κ below.

Definition 3 (cumulative hierarchy). $V_0 = \emptyset$; $V_{\alpha+1} = \mathcal{P}(V_\alpha)$, the set of all subsets of V_α; and $V_\lambda = \bigcup_{\alpha<\lambda} V_\alpha$ for a limit ordinal λ. Every set belongs to some V_α.

Definition 4 (inaccessible cardinal). An uncountable cardinal κ is *inaccessible* if it is *regular* (a union of fewer than κ sets, each of size less than κ, has size less than κ) and a *strong limit* ($2^\lambda < \kappa$ whenever $\lambda < \kappa$).

The two conditions say that κ cannot be reached from below by the two ways of making sets bigger: taking a union of fewer than κ smaller sets, and taking all subsets of a smaller set. $\aleph_0$ satisfies both (a finite union of finite sets is finite, and a finite set has finitely many subsets), which is why the definition asks for an uncountable κ: an inaccessible is to the uncountable cardinals below it what $\aleph_0$ is to the finite numbers.

If κ is inaccessible, V_κ is closed under every construction a type theory needs: for $A \in V_\kappa$ and a family $B(x) \in V_\kappa$ indexed by $x \in A$, the set of subsets of A, the *dependent sum* $\Sigma(x \in A)B(x)$ (the set of pairs (x, y) with $x \in A$ and $y \in B(x)$; when A is a set of tags this is a *disjoint union*, written $\biguplus_{x\in A} B(x)$) and the *dependent product* $\prod_{x\in A} B(x)$ (defined below) are again in V_κ. So V_κ can serve as "the type of all small types". ZFC cannot prove that an inaccessible exists (with one, V_κ would be a model of ZFC, proving ZFC consistent), so the existence of inaccessibles is the extra axiom the target assumes.

Notation. Throughout, $[\![T]\!]$ is the set a type T is interpreted as, and $[\![v]\!]$ the element a value v is interpreted as.

Universes. Fix inaccessibles $\kappa_0 < \kappa_1 < ...$, one for each universe level used. Type_i is interpreted as V_{κ_i}: a type in Type_i is a set in V_{κ_i}. Since $V_{\kappa_i} \in V_{\kappa_{i+1}}$, the rule $\mathsf{Type}_i : \mathsf{Type}_{i+1}$ holds in the model.

Propositions, proofs and proof irrelevance. Fix an object $\bullet$, "the proof". A proposition is interpreted as $\emptyset$ (false) or $\{\bullet\}$ (true), so `Prop` is interpreted as the two-element set $\{\emptyset, \{\bullet\}\}$, and every proof is interpreted as $\bullet$. Any two proofs of one proposition therefore have the same meaning; this *proof irrelevance* is what makes it sound for the calculus to represent every proof by the single value $\star$. A *subsingleton* is a set with at most one element; every proposition denotes one.

Dependent products. A function type $\Pi(x : A).B$ whose codomain is data or a type is interpreted as the dependent product $\prod_{x\in[\![A]\!]}[\![B]\!](x)$: the set of functions f with domain $[\![A]\!]$ such that $f(x) \in [\![B]\!](x)$ for every x.

Products of propositions, and impredicativity. When B is a proposition the product cannot be used as it stands, because it is not a truth value, but it is always one step away from one. Each $[\![B]\!](x)$ is $\emptyset$ or $\{\bullet\}$, so there are two cases. If $[\![B]\!](x_0) = \emptyset$ for some x_0, no function can choose a value at x_0, and the product is $\emptyset$. If $[\![B]\!](x) = \{\bullet\}$ for every x, each $f(x)$ is forced to be $\bullet$, so exactly one function qualifies, the constant function $x \mapsto \bullet$ (when $[\![A]\!]$ is empty, it is the empty function), and the product is $\{x \mapsto \bullet\}$. So the product always has at most one element: it is empty exactly when "$[\![B]\!](x)$ is true for every x" fails. But its one element, when there is one, is a function, not $\bullet$, so the product is a singleton without being the set $\{\bullet\}$. The model therefore interprets a Π into a proposition not as the product but as the truth value the product carries:

$$[\![\Pi(x : A).B]\!] = \{\bullet \mid [\![B]\!](x) = \{\bullet\} \text{ for every } x \in [\![A]\!]\},$$

which is $\{\bullet\}$ when the product is a singleton and $\emptyset$ when it is empty. This collapse loses nothing, and it is forced by proof irrelevance: every proof denotes $\bullet$, so a proof of $\Pi(x : A).B$, for instance a λ, must denote $\bullet$ too rather than the function $x \mapsto \bullet$, and applying such a proof to an argument denotes $\bullet$, which is what $[\![B]\!](x)$'s one element is. Since no rule can look inside a proof, nothing can tell the collapsed meaning from the product it replaces. The result is always $\emptyset$ or $\{\bullet\}$, a member of `Prop`'s interpretation $\{\emptyset, \{\bullet\}\}$, *however large* $[\![A]\!]$ is, even when A is `Prop` itself or a universe. This is *impredicativity*: a proposition may quantify over all propositions, or over all types of a universe, without moving to a larger universe. The set model gets it for free because the size of the domain never reaches the result. The same move is unavailable for data: `Π(A : Type₀). A → A` is a set of functions on all of V_{κ_0}, which is too large to be a member of V_{κ_0} again, so a Π over a universe into data must live in a larger universe.

Least fixed points. An operator Φ on families of sets is *monotone* if $X \subseteq Y$ (pointwise) implies $\Phi(X) \subseteq \Phi(Y)$. Its *least fixed point* is the smallest family X with $\Phi(X) \subseteq X$, the intersection of all such families, which exists for a monotone operator acting inside a fixed set such as V_κ. An inductive type is interpreted as the least fixed point of the operator "one more layer of constructors". In Ochr no recursive occurrence of a type lies under a

Π (fields are first-order), so every element is built by finitely many constructor applications, and the least fixed point is the union $\bigcup_{n<\omega} \Phi^n(\emptyset)$, which stays inside V_κ.

Why a model gives consistency. A model assigns to each type T a set $[\![T]\!]$ and to each well-typed term $t : T$ an element $[\![t]\!] \in [\![T]\!]$; that it does so for every derivable typing is the model's *soundness*. Since $[\![\mathsf{False}]\!] = \emptyset$, no closed term of type `False` can exist, and since the model is built in ZFC with the inaccessibles, the conclusion is relative to that theory. For Ochr, "well-typed term" is replaced by "definition accepted by the checker", and the model's soundness is Corollary 15, which follows from the theorem of Section 4.3 that the checker's steps keep meaning.

4.1.2 The interpretation

Types are interpreted as sets in the way just described, adapting Werner's model of the calculus of inductive constructions (Coq's type theory) and Carneiro's of Lean. As above, $[\![T]\!]$ is the meaning of a closed type value T, and $[\![v]\!]$ of a closed value.

Definition 5 (meaning of types).

- $[\![\mathsf{Type}_i]\!] = V_{\kappa_i}$ and $[\![\mathsf{Prop}]\!] = \{\emptyset, \{\bullet\}\}$.
- An inductive declaration in $\mathtt{Type}_0$ denotes, simultaneously for every value of its parameters, the least family of sets closed under its constructors: the least fixed point of the operator sending a family X to $\bar{a} \mapsto \biguplus_i \{\mathsf{C}_i\} \times \Sigma(g_1 : B_1)...\Sigma(g_k : B_k)$, where each field type is read at the parameters $\bar{a}$ and the earlier fields, with every occurrence $\mathsf{D}\big(\overline{b}\big)$ read as $X\big(\overline{b}\big)$. Non-uniform occurrences make the fixed point simultaneous over the whole family; it is still the least fixed point of a monotone operator on families.

  For example, `inductive List (A : Type) := Nil | Cons(h : A, t : List(A))` has, at a parameter value T, the operator

  $$\Phi(X) = \{\mathsf{Nil}\} \uplus (\{\mathsf{Cons}\} \times [\![T]\!] \times X),$$

  writing $\mathsf{Cons}(a, l)$ for the triple (Cons, a, l). Its iterates from $\emptyset$ are $\Phi^0(\emptyset) = \emptyset$; $\Phi^1(\emptyset) = \{\mathsf{Nil}\}$, since `Nil` has no fields and a `Cons` needs an element of $X = \emptyset$; $\Phi^2(\emptyset)$ adds the one-element lists $\mathsf{Cons}(a, \mathsf{Nil})$ for $a \in [\![T]\!]$; and in general $\Phi^{n+1}(\emptyset)$ is exactly the lists of length at most n, each stage containing the one before (by induction, since Φ is monotone). Their union $L = \bigcup_{n<\omega} \Phi^n(\emptyset)$ is therefore exactly the finite lists of elements of $[\![T]\!]$: a list of length n appears at stage $n+1$, and every stage holds only finite lists. L is a fixed point. An element of $\Phi(L)$ is `Nil`, which is in L, or $\mathsf{Cons}(a, l)$ with $l \in L$, so l is in some stage $\Phi^n(\emptyset)$ and $\mathsf{Cons}(a, l)$ in the next. Conversely an element of L lies in some $\Phi^{n+1}(\emptyset) = \Phi(\Phi^n(\emptyset)) \subseteq \Phi(L)$, by monotonicity. And L is the least such set: if $\Phi(Y) \subseteq Y$, as for any fixed point Y, then every stage is inside Y, by induction on the stage, since $\emptyset \subseteq Y$, and $\Phi^n(\emptyset) \subseteq Y$ gives $\Phi^{n+1}(\emptyset) \subseteq \Phi(Y) \subseteq Y$. So $L \subseteq Y$, and $[\![\mathsf{List}(T)]\!] = L$.
- An inductive declaration in `Prop` denotes the same fixed point, truncated: $\{\bullet\}$ if inhabited, $\emptyset$ otherwise. So $[\![\mathsf{False}]\!] = \emptyset$, $[\![\mathsf{True}]\!] = \{\bullet\}$ and $[\![\mathsf{And}(P, Q)]\!] = [\![P]\!] \cap [\![Q]\!]$.
- $[\![\mathsf{Eq}(A, a, b)]\!] = \{\bullet\}$ if $[\![a]\!] = [\![b]\!]$, and $\emptyset$ otherwise. An equation field denotes the same.
- A function type with borrow parameters $x_i : \&T_i$ for $i \in I$ denotes a subset R (defined next, Definition 6) of $\Pi\big(\overline{d} : \overline{D}\big).\mathrm{Out}_B\big(\overline{d}\big)$, where $D_i = [\![T_i]\!]$ for $i \in I$ and $D_j = [\![A_j]\!]$ otherwise, $\mathrm{Fin} = \prod_{i \in I} [\![T_i]\!]$ is the tuple of final contents of the borrowed places, and $\mathrm{Out}_B\big(\overline{d}\big)$ is $[\![B]\!]\big(\overline{d}\big)$ if B is a proposition, $[\![B]\!]\big(\overline{d}\big) \times \mathrm{Fin}$ if B is borrow-free data, and $[\![T]\!] \times ([\![T]\!] \to \mathrm{Fin})$ if $B = \&T$. For F in this set, fwd_F and back_F are the two components of $F\big(\overline{d}\big)$; $\mathrm{back}_F\big(\overline{d}\big)$ is the *backward function*, mapping the value the caller leaves in the returned borrow to the final contents. A Π-type whose class is "returns proofs" is the exception: as argued in Section 4.1.1, it denotes not this set but the truth value it carries, $\{\bullet\}$ if $[\![B]\!]\big(\overline{d}\big) = \{\bullet\}$ for every $\overline{d}$ and $\emptyset$ otherwise (impredicativity). Its borrowed places need no final contents, since a proof's run is discarded.

  Example, a function that takes a borrow. In-place addition, `def AddM (x : &Nat) (y : Nat) : Unit`, which walks down `*x` to its final `Z` and replaces it with `y`. Its one borrow parameter is `x`, so $I = \{x\}$, both domains are $[\![\mathsf{Nat}]\!] = \mathbb{N}$, and $\mathrm{Fin} = \mathbb{N}$ is the final content of `*x`. The codomain `Unit` is borrow-free data, so a meaning is a function $\mathbb{N} \times \mathbb{N} \to [\![\mathsf{Unit}]\!] \times \mathbb{N}$, and `AddM` denotes $(n, m) \mapsto ((), n + m)$: started with `*x` holding n, it returns `()` and leaves $n + m$ in `*x`. The borrow is thus modelled as passing the place's starting content in and getting its final content back out.

  Example, a function that returns a borrow. `def TailM (x : &Nat) : &Nat`, which returns a borrow of the final `Z` inside `*x`. Again $I = \{x\}$ and $\mathrm{Fin} = \mathbb{N}$, but now $B = \&\mathsf{Nat}$, so a meaning sends the starting content n of `*x` to a pair in $\mathbb{N} \times (\mathbb{N} \to \mathbb{N})$, and `TailM` denotes $n \mapsto (0, v \mapsto n + v)$. The first component, $\mathrm{fwd}(n) = 0$, is what the returned borrow holds when the call returns: the final `Z`. The second, $\mathrm{back}(n) = (v \mapsto n + v)$, is the backward function: if the caller leaves v in the returned borrow before it ends, `*x` finally holds n with its final `Z` replaced by v, that is $n + v$. This backward function is injective (different v give different final contents), which is what the injectivity relation defined next demands of every function returning a borrow.
- $[\![\mathsf{Id}(A, t, u)]\!] = \{\bullet\}$ iff the meanings of the two observations of Section 2.4 are equal.

What R is for. For a function that returns a borrow, the backward function says how the final contents of the borrowed places depend on the value the caller leaves in the returned borrow. The plain product of functions contains every pair $(\mathrm{fwd}, \mathrm{back})$, including ones whose backward function ignores its argument entirely. The model must leave those out, because no Ochr function behaves that way and the checker accepts proofs that rely on it. R is the condition that leaves them out: the backward function of every function returning a borrow, at every type, must be *injective*, so that different values left in the returned borrow give different final contents.

Why every Ochr function has an injective backward function. A function type returning a borrow must have a borrow parameter, so Ochr has no `'static` borrows: a returned borrow is always derived from one of the function's borrow arguments, and points at some position inside the places they lend. When it ends, the value the caller left in it is put back at that position. So the final contents still hold that value at that position, and two different values give two different final contents. For `TailM` above, $\mathrm{back}(n) = (v \mapsto n + v)$: the value left in the borrow becomes the final `Z`'s replacement, and can be read back off the result. A function returning a leaked borrow would be different: Rust allows a `fn(&mut u64) -> &mut u64` to return `Box::leak(Box::new(0))`, a borrow of a fresh place that nobody owns, whose backward function is constant because what is written through it never reaches the caller's places.

Why the model must build it in. The checker relies on it, so a model without it would make an accepted program false. Take

```
def PF (x : &Nat) (e : Id(Unit, *x := 0, *x := 1)) : False := e
```

At its generic call the statement observes the owner of `x`, where the two sides leave `0` and `1`, so it computes to $\mathsf{Eq}(\mathsf{Nat}, 0, 1)$, which is `False`, and `PF` is accepted: writing `0` and writing `1` through a borrow can never be indistinguishable. Now call `PF` with a borrow returned by an unknown function $g : \Pi(x : \&\mathsf{Nat}).\&\mathsf{Nat}$. Its hypothesis says that writing `0` and writing `1` through $g(x)$ leave the same contents, which in the model says $\mathrm{back}_g(n)(0) = \mathrm{back}_g(n)(1)$. If the meaning of that function type contained a g with a constant backward function, this hypothesis would be true for that g, and `PF` would turn it into a proof of `False` that is true in the model, which no sound model allows. So the meaning of a function type returning a borrow must contain only functions with injective backward functions, which is R.

Definition 6 (injectivity relation). For each type A, $R_A \subseteq [\![A]\!]$ is the set of values of A all of whose borrow-returning functions, however deeply nested, have injective backward functions. It is defined by induction on A. For propositions and universes $R_A = [\![A]\!]$. At an inductive type, R holds of a constructor value when it holds of each field. $R_{\&T} = R_T$. For a function type, $F \in R_{\Pi(\overline{x:A}).B}$ iff for all arguments in R, $\mathrm{fwd}_F\left(\overline{d}\right) \in R_B$ and, if $B = \&T$, $\mathrm{back}_F\left(\overline{d}\right)$ is injective.

Making the definition well founded. Read naively, Definition 5 and Definition 6 refer to each other at function types (the meaning is a subset cut out by R, and R a subset of the meaning), and "by induction on A" has nothing smaller to recurse on: a function type's codomain is computed by running code at each argument (a type function such as `Cells(E, n)` is defined by recursion on data), and a type variable $A : \mathsf{Type}_0$ has no structure. Both are resolved as follows, and test 0 checked the resolution on a slice (Section 6.1). A function type means only its good functions, so R at every type is the meaning itself and the two definitions are one; a type variable means a set, all of whose elements are good. Meanings are defined on terms, not on type values, by recursion on the item's position in the program, then the term's structure, then, inside a recursive function, the rank of its decreasing argument; Type_i means V_{κ_i} outright. What remains, beyond the slice, is Q2 (Section 10).

Definition 7 (truth). A closed proposition value P is *true*, $\vDash P$, when $[\![P]\!] = \{\bullet\}$.

Truth is semantic because propositions include Π-types (implication and universal quantification over data, functions and propositions), user inductives in `Prop`, and `Eq` at any type. In particular a stuck equation is true exactly when its two sides mean the same, whatever the checker can compute.

4.1.3 Conditions under which the model is defined

Lemma 8 (the model is well defined). If every inductive declaration satisfies (I1)–(I6) below, every match on a proof satisfies (I7), recursion satisfies (I8), `Eq` satisfies (I9), and borrows are only of Type_0, then Definition 5 assigns a set to every type value formed by an accepted program, in the universe of its sort.

- (I1) *Strict positivity.* The declared type `D` occurs in a field type only as the whole field type, or inside another inductive at a *nestable* parameter (one occurring in that inductive's fields only as a whole field type or at a nestable parameter); never left of a Π, under `&`, or as the argument of a type function.
- (I2) The arguments of an occurrence of `D` never mention `D`.
- (I3) A parameter is never called in a field type; only earlier global type functions are.
- (I4) Parameter types do not mention `D`.
- (I5) Fields form a telescope: a field's type mentions only parameters and earlier fields.
- (I6) Every field of an inductive in `Type₀` or `Prop` lives in `Prop` or `Type₀`.
- (I7) A match on a proof produces data or a type only from a subsingleton (no constructors, or one whose fields are all proofs).
- (I8) Recursion is never on a proof.
- (I9) `Eq` decides only by conversion (`True`) or by distinct constructors of one inductive (`False`); otherwise it is stuck. So an equation at any type, functions and sorts included, adds no rule that could prove `False`.

Proof sketch. (I1)–(I5) make the constructor operator monotone on families, so the least fixed point exists; (I6) keeps it in the sort's universe; (I7) is what makes a match on a proof well defined, since all proofs are • and only a subsingleton's eliminator cannot tell them apart; (I8) because a proof has no structure to measure; (I9) because constructors are injective with disjoint images. Borrows only of `Type₀`: borrowing a universe would put `Π(x : &Type₀)(a : *x). *x` in `Type₀`, which would be impredicative and have no set model. □

> **Quick explainer: strict positivity and W-types.** An inductive declaration is read as "the least set closed under its constructors", and such a set exists only if the type being declared is used *positively* in its fields. A field that is a function *out of* the type breaks this, and no set fits the declaration. In Ochr, with `inductive Bad := MkBad(f : Π(x : Bad). Empty)`:
>
> ```
> def L (b : Bad) : Empty := match b { MkBad(f) => f(b) }
> def bad : Bad := MkBad(λ(x : Bad) : Empty => L(x))
> def Boom : False := absurd(L(bad))
> ```
>
> `L` applies `b`'s own field to `b` itself; `L(bad)` would loop forever, but `Boom` is typed by its callee's statement alone and proofs are not run, so it would be a closed proof of `False`. (Before the positivity rule existed, the checker accepted exactly this.) Linearity does not stop it: `f` is used once, and `b` is consumed only once, by the call, because the match does not consume it (it names `f` as a sub-place of `b`). A language where a match consumes its scrutinee, such as Bend, must rebuild the value and write `f(MkBad(f))`, using `f` twice, which affinity rejects; Ochr's matches do not consume, so it needs positivity instead. *Strict positivity* is the syntactic rule that rules this out: the type being declared appears only as a whole field type, or nested inside another inductive that itself uses it positively, never left of an arrow. A *W-type* is an inductive with a field that is a function *into* the type, such as `Tree := Node(children : Π(n : Nat). Tree)`, an infinitely branching tree. That is strictly positive and Lean allows it, but its least fixed point is reached only after transfinitely many stages of the construction, which the model must have room for. Ochr forbids any function-typed field, so it has no W-types, and its fixed points are reached after ω stages (Section 4.1.1). This is one reason Ochr may be weaker than Lean (Q1).

Status. paper argument. Every condition is enforced by [Ind-decl], the match rules and [Rec]; a probe of each condition against the checker found none missing. One condition is enforced more strictly than (I9) needs: equation fields are required to be over first-order data, which a type parameter instantiated at a function type bypasses harmlessly.

4.2 Conversion is sound

Lemma 9 (conversion respects meaning). If $v \equiv w$ (definitionally equal) at a state Ω reached by the checker, then $[\![v]\!]_\gamma = [\![w]\!]_\gamma$ for every assignment γ of Ω (Definition 10). In particular, convertible closed values (no free variables and no unknowns, Section 2) have equal meanings, and convertible propositions are equally true.

The argument is one principle per conversion rule. Every principle reads the values under an arbitrary assignment, so it speaks of every element of every type, not only of values the machine can build.

- (V1) Normal forms are compared structurally; the meaning of a normal form is a function of its parts.

- (V2) A constructor value means its tag and fields; the parameters it records are fixed by its type and never read, so ignoring them is sound.
- (V3) A sealed program means the meaning of its program (Definition 11), so two with the same text have the same meaning under every assignment, and refinement substitutes into both alike.
- (V4) A Π-type means its family of parameter types and codomain over every argument. [Conv-pi] compares the two at the generic arguments of the first, which are fresh unknowns. An assignment extended to those unknowns can give them any element of the parameter types' meanings, so, by Theorem 14 applied to the evaluation of the parameter types and the codomain, equal meanings at the generic arguments under every extension are equal meanings at every argument. The class and the borrow flag are part of the meaning: they decide which functions inhabit the type and how their calls close off.
- (V5) A function value means a function on its annotated domain, with the backward functions of Definition 5. [Conv-fun] compares the observations of the two generic calls, which run each function's body on fresh unknowns. By Theorem 14 applied to those runs, the meaning of an observation under every extension of γ is the function's result and final contents at every argument, since the fresh unknowns range over every element of the domain, non-computable functions included. So convertible observations mean equal functions.

  Why: nothing is compared with a run at an instance: the generic call's own meaning, under every assignment, already covers every argument.
- (V6) Conversion is least: a comparison that needs itself answers "not convertible". Read coinductively, any two functions stuck at their generic calls would be identified, and `J` along that proves `Eq(Nat, S(Z), Z)`.
- (V7) The unit laws and η at `Unit` are equalities of sets: $[\![\texttt{True} \wedge P]\!] = [\![P]\!]$, and `Unit` has one element.
- (V8) `Eq` computes by meanings: reflexivity by (V1)–(V7), injectivity and disjointness by (V2), and an equation over $\bot$ stays stuck. `J` returns its argument only at convertible endpoints, where it is the identity on meanings.
- (V9) A value that contains $\bot$ is convertible with nothing, itself included ([Conv-bot], D83). $\bot$ means no information, so two values that hold it may stand for different values, and identifying them is not monotone; without this rule, conversion's identity test gave a closed proof of `False`, through conversion itself and through `trans` (Section 9). [Rewrite]'s search for occurrences, a separate code path, has the counterpart: it finds no occurrence of a value holding $\bot$. The cost is one known incompleteness: a type holding $\bot$ is not convertible with itself (`SelfBot`, a `P → P` for such a `P`).

Conversion need not be stable under refinement: it is a check made at one state, and Theorem 14 carries its conclusion through every later step.

An induction shared with Theorem 14. (V4) and (V5) use Theorem 14 on runs that conversion makes, and Theorem 14 uses this lemma at every leaf. Both are proved by one induction on the checker's derivation, which is finite because the checker accepted: the runs a conversion check makes are parts of that check's derivation.

Status. paper argument. Every rule is enforced by the checker's conversion; 35 probes, among them transport along true equations between in-place programs, behave as predicted.

4.3 Meaning preservation

Reminder.
- The checker checks a definition d_k once, at its *generic call*, where each parameter is bound to a fresh abstract value ([Def], Section 2.6). A match on an unknown *splits*, checking each arm in its own *world* ([Split], Worlds); a call that gets stuck on an unknown is *closed off* into a sealed program $\lceil t \rceil$ ([Close], Section 2.3).
- $[\![T]\!]$ is the set a closed type T means in the model (Definition 5), and R the injectivity relation (Definition 6).
- The check of d_k builds a finite *case tree*: a node for each split, with one child per arm, and a leaf where the body ends.

4.3.1 The idea

The checker never runs a definition on real inputs. It runs it once, on unknowns, and accepts it if every leaf of the case tree has the goal's type. The plan does not run the definition a second time on concrete inputs. It gives each state of that run a meaning, as a function of what the unknowns stand for, and compares the run with the *reference run* of the same term: the meaning of running it, which applies the meanings of functions

at calls instead of running their bodies (Definition 13). Every step of the checker keeps that meaning, or loses only information. At a leaf the checker has established that the result has the goal's type, so under every choice of what the unknowns stand for, the result's meaning lies in the goal's. Since the unknowns range over every element of their types, that is exactly the statement that the definition means an element of its type.

Why: comparing the checker's run with a second, concrete run would need a relation saying how their states correspond, and every way the two runs can differ (a borrow ended early on one, a hole on the other, a renamed unknown) would be a case of that relation. Comparing with meanings needs none: the "other side" is mathematics, which has no states.

Quick explainer: symbolic execution and its soundness. *Symbolic execution* runs a program on unknowns instead of values. Where the program branches on an unknown, the executor follows every branch, recording on each what it learned (a *path condition*, such as "$\sigma = 0$"). Each symbolic state stands for many concrete situations: one for each choice of values for its unknowns that satisfies its path condition. The executor is *sound* if each symbolic step maps every situation its state stands for to one the next state stands for. The statement is one-sided: a symbolic state may stand for *more* situations than can really arise (an *over-approximation*), which only makes the executor more cautious. No second run is involved; the comparison is between one step and what the step means. Aeneas, the Rust verification tool whose borrow model Ochr's machine follows, proves its symbolic semantics sound against its concrete one in this style. Here the situations are not machine states but set-theoretic meanings, which is what lets them include non-computable functions.

4.3.2 Assignments and the meaning of a state

Definition 10 (assignment). Let Ω be a state the checker reaches while checking a definition. An *assignment* γ of Ω gives each abstract value σ of Ω an element $\gamma(\sigma) \in [\![\Delta(\sigma)]\!]_\gamma$, in the order the unknowns were created (a type in Δ may mention earlier unknowns, so its meaning depends on γ at them). No separate condition of injectivity is needed: a type means only its good elements, so R is the meaning itself (Section 6.1). It must satisfy the *path condition* of Ω: for each generalisation record $n := \sigma$ in force in the current world, $\gamma(\sigma) = [\![n]\!]_\gamma$. Refinements made by a split or an η-expansion substitute a constructor value for σ throughout the state, so they need no condition. An assignment of d_k's generic state is a choice of arguments for d_k: one element of each parameter's type, including the initial content of each borrowed cell.

Nothing requires $\gamma(\sigma)$ to be computable. For $\sigma : \Pi(n : \mathsf{Nat}).\ \mathsf{Nat}$ it ranges over all functions $\mathbb{N} \to \mathbb{N}$, the characteristic function of the halting problem included; for $\sigma : \mathsf{Type}_0$, over every set in V_{κ_0}; for $\sigma : \mathsf{Prop}$, over $\emptyset$ and $\{\bullet\}$.

Why: the generic call binds each parameter to a fresh unknown, and an assignment says what those unknowns stand for. Ranging over every element of the model's sets, not only over values the machine can build, is what makes the conclusion a statement about the model's Π, which quantifies over every set function. A proof that ranged only over values the machine can build would not reach the model's statement (Section 4.3.8).

Quick explainer: information orders. An *information order* compares two descriptions of the same thing by how much they say. A description that leaves something open is *below* one that fills it in, and nothing it does say may contradict the other. The least element says nothing at all, and is often written $\bot$, which is why the moved-out marker can be read as it. A step is *monotone* if more information in gives more information out, or the same: a monotone step never turns "unknown" into a definite answer that more information would contradict.

Definition 11 (meaning of a state). Under an assignment γ:

- σ means $\gamma(\sigma)$; $\star$ means $\bullet$; a constructor value means its tag and its fields' meanings; a type value means its set (Definition 5), read at the meanings of what it embeds.
- A sealed program $\lceil t \rceil$ means $[\![t]\!]_\gamma$, the meaning of running its program, its reference run (Definition 13).
- A closure means a function on its annotated domain: at each element of its parameter types' meanings, the meaning of its body with the parameters bound to that element. The function may be partial, taking an error value where the body goes wrong; no accepted derivation reaches such a point. A recursive closure, or a recursive global, has a meaning by *guarded* recursion on the rank of its decreasing argument: a recursive call whose decreasing argument does not drop in rank means the error value. So every `fix` has a meaning, checked or not; passing [Rec] is what makes the guard never fire, which is part of membership.
- A state with live borrows is read under a *future* φ, which gives each live borrow's label ℓ its *final content* $\varphi(\ell)$, an element of the meaning of the borrow's content type. A loan of ℓ, at a place that lent its content, means $\varphi(\ell)$: what the place will finally hold. A place holding $\mathrm{borrow}_\ell w$ means w's meaning, its *current* content, which is what a read through it sees. A sealed fill means its program, run with each hole read from φ. So a state means a function of a future, as a function returning a borrow means a backward function (Definition 5). We write $[\![\Omega]\!]_\gamma(\varphi)$ for the meaning of Ω under γ and φ.
- An open dependent value means the raw tuple of its fields' meanings, which need not lie in the set its telescope describes; [Repack] is where membership is restored.
- $\bot$ means *no information*: the least element of the order of Definition 12. The model ignores linearity otherwise: a move makes a place inaccessible to the program, but changes no mathematical value.

A *configuration*, a state together with a term still to run, means the reference run of the term from that state (Definition 13).

Why two readings. A place that lent its content will be given back the borrow's final content, so it means that; a place holding the borrow can still read and write it, so it means what is there now. [End] connects the two: it fixes the borrow's final content at its holder's current content, and forgets the holder. The Lean encoding of the state (Section 6.4) found this shape necessary: one function from the final contents to the resolved contents cannot express a read through a borrow.

Why: every position of a state the checker builds now has a meaning that depends only on γ and on the final contents of the borrows still live. The fills with holes (with `Pick(n, x, y) := match n { Z ⇒ x, S _ ⇒ y }` at an unknown `n`, the hole of its returned borrow sits in the fills of both lent places) are ordinary functions of the hole; reading a place whose meaning depends on a live hole ends that borrow, which fixes the hole at the borrow's current content.

Why $\bot$ means no information. $\bot$ could instead mean the value that was moved out, which would make moves invisible and need no order. Two tests ruled that reading out (Section 6). A stuck block takes a place by move when some arm moves it, even if another arm *writes* it, so the block's $\bot$ would have to mean a value the writing arm contradicts. And the checker's $\bot$ does not record what was moved, so the meaning of a state would need ghost data that the state does not hold. Read as no information, both problems disappear, at the price of the order below.

Definition 12 (less informative). A state meaning m' is *less informative* than m, $m' \sqsubseteq m$, when every reading of m' is a reading of m: for every future φ' there is a future φ such that $m'(\varphi')$ and $m(\varphi)$ agree at every position where $m'(\varphi')$ says something (is not no information). A step is *exact* when it keeps the meaning; an *end* when it ends a borrow, which narrows the futures to those in which the borrow ends with its current content and forgets the holder; and an *over-approximation* when it forgets a place, as a stuck block's capture by move does where the arm the assignment selects would keep it. Ends and over-approximations are less informative.

4.3.3 The theorem

Definition 13 (reference run). The *reference run* Run (t) of a term t sends the meaning of a state, under an assignment and a future, to the meaning of a result and of a new state, or to the error value. It is defined by structural recursion on t, inside the model's recursion on program position and decreasing argument, with the machine's rules except four:

- every call applies the callee's meaning (an earlier global's, a parameter's under the assignment, or a closure's) and never runs its body; for a callee returning a borrow, each lent place means its component of the backward function at the returned borrow's final content;
- before a place is read, borrowed or matched, it ends exactly the borrows whose final content the place's meaning depends on, and nothing else;
- a match on data reads the tag of the scrutinee's meaning, which under an assignment is always a constructor, so nothing is stuck and nothing is split;
- `Id`, `Eq` and `J` mean as in the model, and statements run on a copy that is thrown away.

A function's meaning is read off the reference run of its body in a fresh frame, with a cell holding each borrowed argument. That this meaning is compositional, a call changing only its result and its borrowed places, is the frame property (Section 4.3.6). On the slices, test 0 gave the pure part of this definition and test 2 the state, `Id` and the steps of a higher-order slice (Section 6); beyond them, Q2.

Why: the reference run is the denotation of the term as a state transformer, not a second machine run: it never runs a callee's body, it applies the callee's meaning, which at a function parameter ranges over every set function. It is the standard each step of the checker is measured against.

Theorem 14 (meaning preservation, membership and well-formedness). Let $\mathcal{P} = d_1 ... d_m$ be accepted and hole-free, and let the checker, while checking d_k, run a term t from a state Ω satisfying (b) and (c). For every assignment γ of Ω, following at each split the arm whose constructor γ selects, the run reaches a value v and a state Ω', and:

- (a) *Below the reference run.* There is an assignment γ' of Ω', extending γ to the unknowns the run created and satisfying its path condition, such that the meaning of v and Ω' under γ' is less informative than $\mathrm{Run}\ (t)$ applied to the meaning of Ω under γ, and that reference run is not the error value. It is proved one rule at a time: each step is exact, an end the reference does not make, or an over-approximation.
- (b) *Membership.* Every value stored in Ω', at a binding, a temporary or in Δ, has its meaning in the meaning of its stored type under every typed future: $[\![v]\!]_{\gamma'}(\varphi) \in [\![T]\!]_{\gamma'}$ for every future φ giving each live borrow a final content of its type. (Since a type means only its good elements, this includes the injectivity of every backward function, R of Definition 6.)
- (c) *Well-formedness.* Ω' is well formed (Section 4.3.4).

(a), (b) and (c) are proved together, by induction on the checker's derivation for d_k, inside an outer induction on d_k's position in $\mathcal{P}$ and the size of its decreasing argument under the assignment.

Why: (a) says the checker never computes anything the meaning of the term does not, only possibly less; (b) says it never stores a value in a slot its type does not describe, whatever the live borrows write there later; (c) says the borrow discipline that gives the state a meaning still holds. None of the three suffices alone, as test 1 showed: of the five known proofs of `False`, part (a) alone catches one (Section 6).

For a split, every assignment has an arm. An assignment gives the scrutinee σ an element of $[\![\mathsf{D}(\overline{a})]\!]_\gamma$, which is a disjoint union of its constructors' field tuples (Definition 5). So exactly one arm's constructor matches, and γ extends to that arm's fresh field unknowns by the element's fields; the arm's refinement $\sigma := \mathsf{C}_i(\overline{\sigma})$ is then a substitution that changes no meaning. A split on a sealed program n first generalises it to a fresh σ with the path condition $\gamma(\sigma) = [\![n]\!]_\gamma$, which γ satisfies by taking that value. So the case tree covers every assignment, and no step needs a machine value with a head constructor, which a non-computable witness does not have (Section 4.3.8).

Where (a) needs (b). Most steps keep meaning by computation alone. Two kinds replace a value by a canonical one: a value whose declared type is a proposition becomes $\star$, and a value whose declared type reads as `Unit`, or as a one-constructor type (η), becomes `()` or that constructor. Those keep meaning only if the value lies in its type's meaning: $[\![v]\!]_\gamma \in [\![P]\!]_\gamma \subseteq \{\bullet\}$, or in the one-element set $[\![\mathsf{Unit}]\!]$. For a stuck call that becomes `()` at a declared `Unit`, that is $[\![f]\!]_\gamma(\overline{a}) \in [\![\mathsf{Unit}]\!]$, membership of f's meaning in its Π-type. That comes from the outer induction for an earlier global; from the outer induction at a smaller argument for a recursive call, since [Rec] made its decreasing argument a strict part of the entry value, and the strict-part relation on a least fixed point

is well founded (Section 4.1.1); from (b) at an earlier step for a closure built earlier in the run; and from γ itself for a function parameter.

Where (b) and (c) are earned. (b) and (c) are not consequences of (a); they are established at particular steps, and the checker's *checks* are where. Membership of a value that leaves an open dependent value is established by [Repack] at every whole-again point, under every future of the holes still lent from it; no stored type depends on a hole, since the one type computed over a lent field, an open value's, quantifies over the hole (D64 amended, D73; test 2), and the check of a lent part is sound as implemented (Section 6.3); membership of a block's or a `J`'s sealed result by the return check at the end of each arm; membership of a recursive function, that its guard never fires, by [Rec]; membership of a function returning a borrow, that its backward function is injective, by an invariant that every hole is placed injectively in the state, kept by [Access], D67, D65 and (C8) (test 2); and membership of a returned borrow's forward value by the requirement that a returned borrow be whole. A check that fails produces no derivation, so only its accepting direction carries an obligation, and that obligation is exactly the part of (b) or (c) the check is there to establish.

Why this is ordinary induction. Every appeal is to an earlier step of the same derivation or to the outer hypothesis. There is no separate typing invariant to establish first: (b) is that invariant, proved step by step alongside (a). Nor can a step meet code that was never checked: a sealed call of an unchecked global could re-run to anything (an unchecked `F (n : Nat) : Nat := Nat` re-runs to a type at a refined argument), but while d_k is checked every global is $d_1, ..., d_{k-1}$, checked and covered by the outer induction, or d_k itself at a smaller argument.

Corollary 15 (an accepted definition means an element of its type). Under Theorem 14 and Lemma 9, for every assignment γ of d_k's generic state, the meaning of d_k's body (its result and the final contents of its borrowed cells) lies in $[\![G]\!]_\gamma$, its goal. So $[\![d_k]\!]$, the function sending each choice of arguments to that meaning, lies in the meaning of d_k's declared Π-type.

Proof sketch. Follow the case tree from the root under γ, using (a) at each step: every assignment reaches one leaf (Ω', v, A), with an extended assignment γ'. By (b), $[\![v]\!]_{\gamma'} \in [\![A]\!]_{\gamma'}$. [Def] checked $A \equiv G'$, the goal refined along the path, so by Lemma 9 $[\![A]\!]_{\gamma'} = [\![G']\!]_{\gamma'}$, which is $[\![G]\!]_\gamma$ because the path's refinements are substitutions and its records are γ''s path condition. By (a), the leaf's meaning is less informative than the reference run of the body, which is d_k's meaning at the arguments γ chooses; but the result and the borrowed cells' final contents are whole at a leaf ([Return]), so on them "less informative" is equality. □

Corollary 16 (consistency). Under Theorem 14, Lemma 9 and Lemma 8, Target A holds.

Proof sketch. A parameterless definition c is checked from the empty state ([Const]), which has exactly one assignment, the empty one. By Corollary 15, $[\![c]\!] \in [\![T]\!]$ for the value T of its type. If T computed to `False`, then by Lemma 9 $[\![T]\!] = [\![\mathsf{False}]\!] = \emptyset$, which has no element. □

4.3.4 What the checker must satisfy

The theorem asks four things of the checker. None asks that a run make progress or terminate; consistency needs neither.

Well-formedness, part (c). Backward functions mean the right thing only under the borrow discipline, so Definition 11 is defined only for states satisfying (WF1)–(WF6) of Section 2.2 (each label has at most one borrow; loans only where closing off allows; no cycles among borrows; closures capture no borrow; arguments exclusive at call points; bindings hold values of their types) and

- (WF7) `&` is applied only to types in Type_0, so that a borrow means forward and backward functions on data.

[Rec] is not among them: guarded recursion gives every `fix` a meaning, checked or not (test 0), and passing [Rec] is what makes the guard never fire, which is membership. The preservation of (WF1)–(WF6) is not yet proved; test 2 assumed it, and found that the preservation of (WF2) needs (WF3). (WF5), loan-free arguments at a call, is not only a runtime matter: with a live loan in an argument, a read now would depend on a write made later, so the meaning of a state needs it.

Monotone computing steps. Steps that over-approximate (a stuck block's captures by move, a capture split less finely, an early end) are harmless only if no later step turns the missing information into a definite answer

that the full information would contradict. So every step that *computes* on a state must be monotone in the order of Definition 12: `Eq` and `J` computing to `True`, `False` or a value, a match choosing an arm, [Close] choosing its row, conversion, which decides that two values are equal, and [Rewrite]'s search for the occurrences of a side, which decides that a subterm equals it. Test 2 checked `Eq`, `J`, matches and [Close]'s rows. The checker has the instances for them: an equation with `⊥` on either side stays stuck rather than computing (a test, `MovedBothSides`, guards it), reading `⊥` is an error, and an observed owner holding `⊥` gives a stuck equation. For conversion and [Rewrite] it has [Conv-bot] (D83, (V9)): a value containing `⊥` is convertible with nothing, itself included, which also covers `trans`, and [Rewrite] finds no occurrence of a value holding `⊥`; without them, identifying two values that hold `⊥`, which may stand for different values, gave a closed proof of `False` (Section 9). So every computing step is monotone over `⊥`. The generalisation records, which also compare sealed programs by text, are safe: `⊥` reaches a data-valued sealed program only inside an erased statement. Only computing steps need monotonicity; decisions need only the weaker conditions below.

Decisions, by kind. A decision need not come out the same on the generic call and at every instance. It must fall into one of four kinds, and the census of Section 6 sorted every one:

- *same*: every outcome keeps the meaning, possibly given (b). `η` at `Unit` and at one-constructor types (the type has one element); a proof's value being `⋆`; `Eq` and `J` computing; [Split], records and re-normalisation.
- *aligned*: the outcomes differ in meaning, so the model must make the same choice, and it can because the decision reads only syntax and declared types, which the model's meaning of code reads the same way. Whether a term is erased (run in place, or on a private copy that is thrown away, discarding writes through borrows); a function type's class; whether a statement argument runs in place; [Close]'s row. A binding's class is one, read from syntax since D77.
- *less informative*: copy or move, and a stuck block's captures by move and splits. Harmless given monotone computing steps.
- *check*: only accepts or rejects, so only the accepting direction needs an argument (above). Some checks establish (b), some (c), and some protect only runtime safety, which consistency does not need but type safety (Target B) does: the requirement that a borrow be whole when it is passed to a call that closes off. (That a returned borrow be whole is membership: a returned borrow holding `⊥` has a forward value with no meaning.)

Five lemmas behind "same". The decisions of kind *same* rest not on one argument per site but on five lemmas: membership, part (b); the soundness of the function that reads a value's own type (`valType?`: the meaning of a value lies in the meaning of the type it reads, under every assignment), used where a type is read off an embedded value; *footprint coverage*, that the places `Id` observes include every place either side may write, under every assignment, with the extra owners at a call site handled by the injectivity of contexts (a context holding a loan is injective in the loan's final value, which test 2′s invariant that holes are placed injectively provides); conversion soundness (Lemma 9); and well-formedness, part (c).

4.3.5 The lemmas each step uses

The step lemmas, one per checker rule, are where the principles of Section 5 are used, read as statements about meaning:

- worlds (X1)–(X4): leaving an arm drops every fact that depends on the arm's path condition, so the assignment that held in the arm's world is never asked to hold elsewhere; keeping such a fact is how `EtaBoom` proved `False` (Section 9);
- closing off (C1)–(C9): [Close] is exact by definition, because a sealed program means its program, and (C1) is what makes the text rebuild the call;
- locality (L1)–(L7), read as the compositionality of the meaning of code: a call's meaning changes only its result and its borrowed places (the frame property, proved on the heap slice as `Ochr.Heap.frame` and `call_effect`);
- observation (O1)–(O5): `Id`'s computation to a conjunction over the footprint is exact by footprint coverage;
- erasure and captures (E1)–(E5), (K1)–(K3): erased runs keep meaning because their copies are discarded, and captures are over-approximations in the sense of Definition 12;
- wholeness (W1)–(W2): every point where a value passes to code relying on its type is a point where (b) is established for it;

- recursion (R1)–(R6): membership of recursive functions (the guard never fires) and the outer induction.

4.3.6 From meanings to type safety

Type safety (Target B) reuses the route. The step lemmas of Theorem 14 are untyped: they speak of every state the checker reaches, and a closed run is such a state with no unknowns, so they cover concrete runs as well. And the reference run of an accepted definition never goes wrong (Theorem 14, part (a)). What remains is to connect the erased run with the reference run. Test 3 (Section 6.5) worked this out on a slice:

- (a) *Error reflection splits by kind of error.* The machine's 140 error sites fall into four families.
  - *Value errors* (11 sites: no such place, not a function, a value no arm matches) reflect to the reference run: at a closed state a value means itself, so the machine's test of its shape fails exactly where the reference run's does. Following * from a value that is not a borrow is the exception, since a borrow means its content; it is excluded by typing, which the runtime invariant must therefore include.
  - *Borrow errors* (18 sites: moved out, a borrow ending or returned partly moved, a dying value borrowed by one in flight) also reflect to the reference run, using the injectivity of backward functions, except at *one-element types* (`Unit`, `CellsEnd`). There the machine can hold a loan that no meaning depends on: in `UseDirect (u : Unit) : Unit := (let r = &u; let z = u; *r := (); ())` the machine ends `r` when it reads `u` and then errs at `*r := ()`, while the reference run, whose `u` means `()` in every future, ends nothing. Those errors reflect instead to the checker's derivation, by a lemma about borrow structure only: at corresponding points of the closed run and the derivation, the same places are moved (exactly, under D84), every borrow the run has ended the derivation has ended, and every live loan of the run has a corresponding loan or hole in the derivation. The lemma is sketched by cases, with no counterexample; the checker rejects `UseDirect` for exactly this reason.
  - *Static and typing errors* (93 sites) are unreachable: the erased run does no typing and closes nothing off, and every check on syntax passed when the definition was checked. *Internal assertions* (18) are invariants.

  paper argument checked in Lean operation by operation, not yet over a whole run.
- (b) *Runtime `J` is its transported term* (D85). The checker computes `J` only at convertible endpoints, and a closed run can reach a `J` whose endpoints have equal meanings but are not convertible: with `f = λ(n : Nat) : Nat => n` and `g = λ(n : Nat) : Nat => match n { Z => Z, S m => S(m) }`, both the identity on $\mathbb{N}$, a cast along a true `Eq(Π(n : Nat). Nat, f, g)` is stuck. In the erased runtime, $\mathtt{J}(A, a, b, P, h, t)$ evaluates to t unconditionally: this is what erasure means, since the endpoints, motive and proof are statements the erased program does not have; it is sound given membership, since true proof parameters make the endpoints' meanings equal (Lean: `castRT_safe`); and it reads nothing. The checker keeps its rule everywhere, its untyped runs included, since those run sealed programs' re-runs and callee bodies. The checker's rule would get stuck at runtime on two encodings of one function (Lean: `cast_fn_counterexample`). Consistency needs nothing here: truth is semantic, and a stuck `J` means transport, the identity at endpoints with equal meanings, which membership of the equation's proof guarantees. This is D85, on branch `runtime-j`, not yet merged.
- (c) *Erasure holds by construction.* A statement's run leaves no trace (it runs on a discarded copy); no runtime step reads a statement's value except as $\star$ (proofs by declared sort, D55; matches on proofs by declared type, D45, with (I7); classes by syntax, D54, D77; no type values in data, (I6)); and calls through function values erase alike. Runtime `J` is its one extra clause. paper argument.
- (d) *Termination.* Recursion by name terminates: [Rec] makes each recursive call's argument a component of the entry value, so of smaller rank, and guarded recursion's guard never fires. Calls through function values are not covered by that measure: a closure passed in, or stored in data, can be called from anywhere, and a function's meaning (its graph) bounds no number of steps. They need a *realizability* relation in the style of Tait and Girard, between runtime values and meanings, defined by the model's recursion, at a type variable over pairs of a set and a relation; it is predicative, since proofs are erased. Not yet built: this is Q4. That is why Target B asks for arguments built by accepted code.
- (e) *Well-formedness at runtime* is the checking-time invariant with no inert loans, shared with consistency (Section 4.3.4). The Lean check proves (WF2), (WF3), and that no borrow sits inside a borrow's content, preserved by [End], [Access], [Drop] and a call; not yet a borrow, assignment or move at a field path, nor

(WF1). The injectivity of backward functions is needed for type safety as well as consistency: the reflection of [Drop] rests on it.

Open lemmas for Target B: the borrow-structure lemma, sketched; realizability, for termination through function values; typing in the runtime invariant; error reflection over a whole run, not per operation; and well-formedness at field paths, with (WF1).

The frame property is a lemma, not a target. Both targets use it: for consistency it is the compositionality of the meaning of code (a call's meaning changes only its result and its borrowed places), and for type safety the fact that a call's effect at runtime is its isolated run, plugged back into the caller.

Adequacy in the narrow sense is the last clause of Target B: a closed run's result means what the reference run says. With it, an accepted `Id` lemma makes its two observations equal at runtime. That the two terms can then be swapped in every context is more, and is Target C.

4.3.7 `Id` adequacy

Target C says that terms proved equal by `Id` are interchangeable. Neither of the other targets implies it. Target A needs only that the checker computes `Id`'s meaning soundly, and that meaning can be weaker than substitution: `Id` compares the results and the observed owners, not everything a later context could notice. Target B does not look at `Id`'s meaning at all.

The new piece is a context lemma: what a context can observe of a term is exactly what `Id` compares. Given it, a proved `Id` makes the two terms indistinguishable to every continuation, and Target C follows from Theorem 14 and Target B. The audit of Section 6.6 lists every channel through which a context can observe a term and compares the list with `Id`'s footprint.

Two known gaps. Neither is a proof of `False`: in both, `Id`'s meaning in the model is computed soundly, and what fails is substitution.

- *Moves.* `Id` does not compare a place that either side leaves moved, so two terms that move different places, such as `F(x)` and `F(y)`, can be proved equal while a continuation that reads `x` tells them apart. D84 (drop at the join), not yet merged, closes this.
- *Where a borrow points* (`RePoint`, accepted on main at `a2861cbf`). `def RePoint (x z : Nat) (y : &Nat) : Id(Unit, (y := &x; ()), (y := &z; ())) := refl`. The resolved owners are unchanged on both sides, so `Id` holds; but a continuation `*y := 7` writes `x` after one side and `z` after the other. So the observation principle that where a borrow points does not matter, only resolved contents (O2), is false under substitution. The likely fix is to observe each borrow variable a side assigns by writing a fresh σ_w through it, as [Obs-borrow] already does for a borrow-typed result.

4.3.8 Challenges a proof must meet

Three independent referees, on 2 November 2026, searched for obstructions to proving Ochr consistent. None found consistency impossible; each found a challenge any proof must meet, and the plan meets each as follows.

- *Truth and proof must meet.* Truth is defined in the set model, which quantifies over every set function, so a proof that reasons only about machine values does not reach it. Ochr can state "f decides halting": with `Prog` a type of two-counter machines, `ExW(Prog → Nat, Decides)` is true in the set model, witnessed by the characteristic function of the halting set, but by no machine value, and an accepted lemma `L3` derives excluded middle for halting from it. A proof over machine values breaks on it: matching on the hypothesis binds the decider to an unknown σ_w, and splitting the stuck $n = \lceil \sigma_w(P_0) \rceil$ needs a head constructor no machine value supplies. In the plan, σ_w ranges over every function, the halting decider included, and n is generalised to a fresh σ_n with the path condition $\gamma(\sigma_n) = \gamma(\sigma_w)(P_0)$; the split on it covers every assignment, since the value is `Z` or `S` of something. Likewise a lemma `M` stating that every decider answers 0 on a program that halts at once holds at every set-theoretic decider by the outer induction at `M`, not vacuously.
- *A closed proof of `False` on main.* A referee found `EtaBoom` (Section 9), and showed that statements relating the generic call to its instances hold only up to an information order. In the plan, `EtaBoom` is a step at which part (a) fails: leaving the arm kept field unknowns whose types were read under that arm's path condition,

so in the sibling arm no assignment exists at all, since one field would need an element of $[\![\mathsf{Void}]\!] = \emptyset$. Over-approximation is built into Theorem 14 through the order of Definition 12.

- *No cycle, and no unchecked code.* A referee found the same gap between truth and proof, the risk of a cycle in which soundness needs a typing invariant that itself needs soundness, and a missing obligation on about 80 unchecked array accesses in the checker. In the plan, truth and proof meet by construction, the typing invariant is part (b) of Theorem 14, proved in the same ordinary induction, and conversion soundness is argued under assignments. The array accesses remain an obligation of the Lean statement, with Q1.

5 The principles the checker enforces

The step lemmas of Theorem 14 use a small number of principles, each derived from a model of one part of the calculus (dependent fields, case splitting, observation, recursion, erasure, closing off, locality), and each enforced in the checker at a single place, so that a rule cannot be forgotten by one of several code paths; Section 4.3 says which step uses which. Every current checker rule corresponds to one of them, and in each area probes looked for a rule the principles demand and the checker lacks. Three such gaps were found on the checker of October 2026, each a closed proof of `False`, and a referee found a fourth, `EtaBoom`, in the worlds principle; all are fixed (Section 9). None of the principles is proved.

5.1 Locality

- (L1) *Names are local.* A running body names only places of its own frame. A closure's captures are copied into its own frame when it is called, and a closure captures no borrow. Global definitions are immutable.
- (L2) *Writes stay in the frame.* A borrow carries its content, so a write through a borrow changes the content inside the borrow held in the running frame; the lender's place does not change until the borrow ends.
- (L3) *Calls exchange only what is passed.* Arguments move into the callee's frame (an owned value is moved out of the caller; a borrow argument carries the caller's loan); the callee returns its result; popping its frame drops its bindings, ending its borrow parameters; [Close] fills only the loans of the call's borrow arguments.
- (L4) *Borrowers sit at or above their loans*, in every run whose effects persist: a loan in frame f is borrowed from f or a frame above it. It follows from no borrows in data, no borrows of borrows, closures capturing no borrow, and [Drop] refusing to let a borrow of a local outlive it. So [Access] and [Drop], which start from a place of the top frame, end only borrowers of the top frame. (L4 fails inside the checker's own private copies, which compute a field's type by binding copies of fields in a scratch frame; those copies are discarded, so nothing leaks.)
- (L5) *Ending is the one channel down.* [End] is how a lender below gets its content back; by (L4) the places a run can change below its frame are exactly the places its frame's borrows were lent from.
- (L6) *Private runs leave no trace.* Statements, and the checker's own scaffolding, run on copies that are thrown away.
- (L7) *Sealed programs are closed, and refinement is not a run.* A sealed program's re-run starts from the empty environment and reads no frame. Refinement substitutes across all frames, but that is the typing judgement acting on the whole state, never the effect of a run.

Enforced: the checker ends a borrower found from a place of the top frame in one function (`endTopLoan`), which asserts (L4) outside private copies; it never fires on the test suite.

5.2 Closing off

- (C1) *Reconstruction.* The sealed text rebuilds the call exactly: the same function value (code and captures), the other arguments as their values at the call point, each borrowed content in a fresh cell. Nothing else of the call point reaches the callee, by (L1) and (L3). So a sealed program is a closed text whose run depends only on what it embeds and on the refinements of the world it runs in.
- (C2) *Projections.* Each output (the result, and each borrowed place's final content) is its own sealed program, and its final read copies. A `Unit`-typed output is `()`.

- (C3) *Holes are parameters.* A returned borrow's final content is a parameter of every fill, because at the generic call it is not decided which borrowed place the returned borrow points into. A fill's run never inspects its hole: an inert loan is stuck at a match and passes through reads and writes unchanged. Filling it is substitution, which then re-runs the fills.
- (C4) *Re-run exactly when an input changes, everywhere.* A sealed program is re-run when a value it embeds is substituted (a refinement, or the fill of a loan) or a neutral its run derives is refined, at every position a sealed program can occupy: the environment, stored types, the types of abstract values, the goal.
- (C5) *No re-closing.* On a re-run the head call is not closed off again, so a sealed program makes progress or stays exactly as it is.
- (C6) *One text, one value.* Within a world, a sealed program's text determines its value (by (C1) and determinism), so two derivations of one text may be identified.
- (C7) *Errors surface.* A re-run that fails is a type error where it was triggered; exhaustion is undecided.
- (C8) *Clean inputs.* Every input is loan-free and whole when the text is made.
- (C9) *Blocks are calls.* A stuck match or `J` is closed off as a call of an anonymous function of its captures, so (C1)–(C8) apply with the captures as arguments.

Enforced: one function makes sealed programs and one re-runs them; substitution and re-normalisation share one traversal of every value position of the state (`mapState`), which closed a gap in (C4) where re-normalisation had skipped the types of abstract values.

5.3 Worlds

Checking a match on an unknown σ of an inductive $\mathsf{D}(\overline{a})$ is elimination of a disjoint union: σ denotes an element of $[\![\mathsf{D}(\overline{a})]\!]$, which is the union of its constructors' field tuples, and each arm is one summand.

- (X1) *Worlds are exclusive.* No fact established while checking arm i that depends on σ being C_i is visible while checking arm j: a refinement, a type refined by it, a generalisation record, a captured value.
- (X2) *Refinement is total substitution.* Entering a world substitutes σ everywhere it can occur (environment, stored types, types of abstract values, goal, values embedded in sealed programs and closures) and re-normalises. A component skipped keeps a stale, pre-refinement value in a refined world.
- (X3) *Records are scoped.* A generalisation record $n := \sigma$ is in force in the world that made it and its sub-worlds (private copies, nested splits), nowhere else: the same text n can denote values of different types in different arms, since it may mention the scrutinee.
- (X4) *The block is checked unrefined.* What follows the match is checked once with σ unknown; its sealed programs resolve when a later split learns σ.

What a world may keep is exactly what holds in every world: fresh-name counters (a counter states no fact) and the types of abstract values created inside the arm (nothing kept names them, since the arm's records are dropped with it). An η-refinement of an abstract value of a one-constructor type is *not* among them. Its constructor follows from the type alone, but its fields are fresh abstract values whose types are read off σ's type in the arm's world, and can mention what the arm refined: a value parameter, or an earlier field of a dependent constructor. Kept, a sibling arm reused fields typed in another world, a closed proof of `False` (EtaBoom, Section 9). So an η-refinement, like a generalisation record, lives until the arm that made it ends, and survives the private copies made within it. *Enforced:* every arm of every split runs inside one combinator (`inWorld`), whose exit restores every component of the state except those listed, so a new component is restored by default.

5.4 Wholeness

A dependent constructor $\mathsf{C}(g_1 : A_1, ..., g_k : A_k)$, with A_j mentioning earlier fields, denotes a *Σ-type*: the set of tuples whose later components lie in sets determined by the earlier ones. A borrow of a field, as in Aeneas (the Rust verification tool whose borrow model Ochr's machine follows), receives back when it ends any value of its type.

- (W1) *Open only while the opener runs.* A value may break its telescope only while the code that opened it runs. Wherever it passes to code that relies on its type, it must be whole: read, moved, copied or borrowed whole;

captured by a closure, Π-type or block; observed by `Id`; at the end of a borrow of it (a borrow parameter's at its function's return); as a result.

- (W2) *Producer obligation.* The generic path treats every neutral of a dependent type as whole: [Repack] does not look inside it, and [Split] splits it by its telescope. So whatever produces such a neutral must have checked whole the value it stands for. A parameter's σ: the caller passed it whole. A field of a refined neutral: whole because the neutral is. A generalised sealed program: inherited. A stuck call's result or fill: the callee's body checked at its return. A fill with a hole: the same, with the lent part unknown. A stuck block or closed-off `J`: each arm, or the transported term, checked at its end. A hole `?`: an axiom, excluded.
- [Return] is the common form: where a checked body returns, its result and every place it holds by borrow must be whole; a part still lent there is any value of its borrow's type.

Enforced: one function forms every block, taking its arms as computations and running them itself, and calls the same return check that function bodies use; so no block can be formed without the check.

5.5 Erasure and captures

- (E1) *Discard.* Every erased term runs on a private copy of the state it would start in, and the copy is thrown away.
- (E2) *Decided by declaration.* Whether a term is erased is read from syntax and declared types. Since sorts are syntactic, "is a proof" is a property of the term itself, so a re-run of a sealed program erases exactly what the direct arm erases.
- (E3) *Only copy-independent facts survive a copy:* name counters, the types of names created in it, generalisation records (keyed by closed program texts, which mean the same in any state), η-refinements (the fields' types are the copy's world's, which the copy shares), checked recursion obligations, caches.
- (E4) *In place means within the statement's copy.* A borrow-typed argument of a statement runs in place, not on its own copy, but the statement call is itself erased.
- (E5) *Reads move inside statements as at runtime,* on the copy only: a side that reads x and then observes x sees $\bot$, and its equation is stuck.
- (K1) *Captures over-approximate.* For each outer place and each arm, a block's capture mode is at least the arm's effect on it, in the order none $\sqsubseteq$ copy $\sqsubseteq$ write or borrow $\sqsubseteq$ move, measured at the granularity of the place moved and net of the arm's own refills; a closure or Π-type in an arm moves what it mentions.
- (K2) *Over-approximation is safe.* A stricter generic call only removes options for the code after the block: a read of a place the block moved is an error, and an equation over it is stuck.
- (K3) *Effects are measured where they happen:* a move inside a nested block is that block's, and a move inside a statement is discarded with its copy. By (E2) a block's re-run decides as the direct arm does.

Enforced: statement positions run through one combinator (`inStatement`) and the checker's scaffolding through another (`privately`); captures are computed by one function (`blockCaptures`).

5.6 Recursion

A definition **fix** $f\left(\overline{x} : \overline{A}\right) : B$ **by** $x_j \coloneqq b$ denotes the function defined by well-founded recursion on the size of the entry value of x_j.

- (R1) *The measure is the entry value, as refined.* Each recursive call's argument (its content, through a borrow) is compared with the caller's entry value as refined so far, whatever the body did to the place since. Strict subterm implies smaller size. Refinement only adds constructors, so a strict subterm at the generic call is one at every instance.
- (R2) *Recursive calls are named.* f is reached only through the name it binds for itself, or a nested function's capture of that name; which calls recurse is a fact about syntax, never decided by comparing function values, which refinement changes.
- (R3) *Check before running.*
- (R4) *One order.* A definition sees only earlier definitions and itself, so every chain of calls is ordered lexicographically by (definition, entry size). A call through a function value is not a recursive call unless

its head is the name; a closure formed in f's body that calls f is checked against f's entry value where it is formed.
- (R5) *Types recurse like terms*, and a statement never mentions the definition being checked.
- (R6) *Runs only unfold.* Untyped runs unfold definitions without the check; they decide nothing about well-foundedness, and a run that does not terminate is exhaustion, never a decision.

Enforced: one function decides whether a call belongs to a recursion entry, by a mark on the binding (`recFrameOf`).

5.7 Observation

- (O1) *Independence.* The two sides of `Id` run from the same state on independent copies, in the same world.
- (O2) *Resolved, not represented.* Observations compare resolved values, so where a borrow sits does not matter; owners are observed, not borrows. This is sound for `Id`'s meaning (Target A), but false under substitution (Target C): a borrow variable that a side reassigns points elsewhere afterwards, and a continuation can write through it (`RePoint`, Section 4.3.7).
- (O3) *Footprint sufficiency.* A position whose resolved content cannot differ between the sides need not be observed. It can differ only if a side writes it, directly or through a borrow, so the owners of every place written or borrowed, and of every borrow variable handed on whole, are observed; a place only read is not (a read that moves leaves $\bot$, which must be refilled before the borrow over it ends, and refilling is a write). Sufficient for `Id`'s meaning; for Target C the footprint must also cover what a continuation can observe: where an assigned borrow variable points, and, until D84, places moved by one side only (Section 4.3.7).
- (O4) *All owners.* A loan can sit in several owned positions (a hole in several fills); each is observed.
- (O5) *One order.* The conjunction is ordered by syntax, the same on both paths, since $\wedge$ is not commutative by conversion.

Footprints at a call site. A footprint's roots are read off syntax, so they are the same on both paths; only the owners can differ. At the generic call a borrow parameter's loan sits in one fresh cell; at an instance it may sit in several owners (O4). Every extra owner's resolved content is a function of what was written through the borrow plus parts no side writes, so if the generic statement proves the written values equal, each extra owner's equation is true too. This is footprint coverage (Section 4.3.4), with the injectivity of contexts.

6 Tests before mechanisation

The route is checked on paper before anything is mechanised, hardest first, following the standing preference to attack first the part most likely to fail. Four tests were planned: tests 0 and 1 independent of each other, the census audit cheap, and test 2 after test 0. Each result is summarised here; the full write-ups are the documents named. All four have passed. The one failure they found is a fifth closed proof of `False`, conversion over $\bot$ (Section 9), since fixed on main by D83. A fifth test, test 3, checked type safety (Target B) on the slice, and found no checker bug.

6.1 Test 0: the model exists, on a slice

What it checks. That the set model, stratified by universe, can be defined without circularity on a slice containing each feature that makes the definition hard: `Nat`; a type function defined by recursion on data (`Cells(E, n)`); a type variable $A : \mathtt{Type}_0$ with R at it; a function type returning a borrow, with injective backward functions; and a Π into `Prop` over $\mathtt{Type}_0$.

What would fail it. A clause with nothing smaller to recurse on, or a meaning that is not a set. Then the plan fails at its base.

Result: the model exists on the slice (`ochr/docs/24`, 6 November 2026; a paper result, most of it also encoded in Lean in `ochr/core/lean/Ochr/Model/Slice.lean`, which builds with no `sorry` and uses only the axioms `propext` and `Quot.sound`). Neither apparent circularity of the definition is an obstacle.
- *A type means only its good elements.* If a function type returning a borrow means only the functions whose backward functions are injective, then R at every type is the meaning itself, and the meaning and R are

one definition. A type variable means whatever set the assignment gives it, and R at it is that whole set: an Ochr program can use a value of a type variable as a function only after transporting along a proof that the variable *is* some Π-type, whose meaning is already a good set. Pairing each type with a separate set of good elements would be not just unneeded but harmful: the Lean encoding found two functions that agree on every good argument yet differ at a junk one, so with pairs two convertible functions would have different meanings and [Conv-fun] would be unsound. With carriers holding only good elements, [Conv-fun] is function extensionality, proved in Lean.

- *Meanings are defined on terms, not on type values.* The recursion is on the position of the item in the program, then the structure of the term, then, inside a recursive function, the rank of its decreasing argument; all three are well founded. A Π-type's codomain *term* is a subterm of the Π-term even though the Π-type value computes the codomain by running code; a computed type $F(a)$ means the meaning of an earlier item at the meaning of a; a type variable is looked up, not recursed into. `Type`$_i$ means V_{κ_i} outright, so there is no universe of codes and no induction–recursion; universe levels matter only for sizes (that a type at level i means a member of V_{κ_i}), which is part of membership, not of the definition. The Lean encoding needs its levels bounded (Q1).
- *A recursive function means a function whether or not it passed [Rec].* Its meaning is defined by *guarded* recursion: a recursive call whose decreasing argument does not drop in rank means the error value. So [Rec] is not needed for the definition; it is what makes the guard never fire, which is part of membership, (b). The defining equation under that hypothesis is proved in Lean for type functions recursive on `Nat`. So [Rec] belongs to membership (Theorem 14, Section 4.3.4).

What it leaves. The meaning of the state (live borrows as functions of their final contents, holes, inert loans, `⊥`) and therefore of `Id` inside types, the next hardest piece, which test 2 took up. Two obligations come with it: a type's meaning never depends on a hole, and every accepted closure returning a borrow has an injective backward function (part of membership). In a typed mechanisation, meaning and membership must be built together, as the Lean encoding already does for borrow-returning functions.

6.2 Test 1: the known proofs of `False` are failed steps

What it checks. For each closed proof of `False` the checker once accepted (Section 9), and for the moved-out equation the test `MovedBothSides` guards, whether a part of Theorem 14 fails at the buggy step and holds once the fix is in. A bug that no part catches would mean the theorem is too weak.

Result: all five are explained, and none escapes (`ochr/docs/25`, 6 November 2026; a paper analysis, the buggy behaviour taken from each fix's commit).

- *A returned borrow of an index field* (`BoomI2`). [Repack] at the function's return checked the owner whole with the lent field read at its current content. Its accepting direction owes (b) for the owner under *every* future of the returned borrow's hole, and the caller writes another value there, at which the record's dependent field no longer fits. (b) fails; the fix, checking the lent part as an unknown of its type, restores it.
- *A stuck `J` closed off unchecked* (`BoomJ`). The fill left in the borrowed place means exactly what running the `J` means, so (a) holds even before the fix. (b) fails for the fill: where the endpoints are equal, it means a record whose dependent field was typed at the old index. The fix, checking each arm of a block at its end, is (b) at a producer of a sealed value.
- *A recursive closure capturing a value it splits on* (`BoomCapSplit`). The split keeps meaning: the refined closure means the same function, so (a) holds. [Rec] identified recursive calls by comparing function values, which the split changed without changing their meaning, so a recursive call went unchecked, and (b) fails: under guarded recursion (test 0) that call's decreasing argument does not drop, so the guard fires and the closure's meaning there is the error value, which lies in no type. The fix, recognising recursive calls by name, is syntax, which refinement does not change.
- *`EtaBoom`.* Leaving the arm $n := \mathsf{Z}$ kept an η-refinement whose fields were typed in that arm. In the sibling arm the field of type `Void` forced every assignment to choose an element of $\emptyset$, so the arm admitted no assignment at all and could prove anything. (a) fails in its coverage part, the case the route predicted. The fix drops the refinement with its arm.

- *MovedBothSides.* At the generic call a stuck block on each side of an `Id` took the same place by move, so the equation between the two `⊥` computed to `True`, though at an instance only one side moves it and the equation is false. Reading `⊥` as no information, that step is not monotone: filling in the two `⊥` differently makes the equation false. The fix keeps an equation with `⊥` on either side stuck.

Five further soundness guards found by ablating the checker fit the same way: a match on `Z`/`S` needs a `Nat` scrutinee (coverage, in (a)); a whole read repacks, a proof field assigned while part of its value is lent is checked under every future, and each arm of a block is checked at its end (all (b)); a borrow is formed only at `Type₀` ((b) for types, and (c)).

What it shows. Part (a) alone catches one of the five. Three are caught by (b): two quantified over the futures of the live holes, one by the guard of a recursive function. In all three the step's (a) holds even before the fix. So the route stands or falls with (b) as much as with (a), and must be presented, as Theorem 14 is, as one induction over all three, never as "each step keeps meaning" alone. Each bug is local to one step and one part of the theorem. The test also favours reading `⊥` as no information: the alternative, `⊥` as the moved-out value, needs the old value, which the checker's `⊥` does not record. A fifth closed proof of `False`, found after this test, fits as well, but as a failure of the monotonicity required of computing steps (Section 4.3.4), not of (a), (b) or (c) (Section 9).

6.3 The census of the checker's decisions

What it checks. Every decision the checker makes, as listed by the two-paths census (`ochr/core/notes/two-paths-census.md`), with every edge case found by ablating the checker and every remaining site that reads a value's own type (`valType?`), sorted into the kinds of Section 4.3.4: same, aligned, less informative, or check.

Result: everything sorts, apart from one item (`ochr/docs/26`, 6 November 2026; read-only, on main with D80 and D81). The audit raised two; a probe has since settled the first.

- `⊥` *must mean no information.* A stuck block takes a place by move when some arm moves it, even if another arm writes it, so `⊥` read as the moved-out value would contradict that arm. Read as no information, block captures, moves and splits are less informative, and the computing steps the audit examined are monotone over `⊥` (an equation over `⊥` stays stuck; reading `⊥` is an error; an observed owner holding `⊥` gives a stuck equation). This is the reading of Definition 11. The audit counted conversion as a check that only rejects, but its accepting direction must be monotone too: after the audit, identifying two values that hold `⊥` turned out to be a fifth closed proof of `False` (Section 9).
- *Same:* the η family, a proof's `⋆`, `Eq` and `J` computing, splits and records, the fill of a lent `Unit`. *Aligned:* erasure, a function type's class, a statement argument's place, [Close]'s row, all read from syntax since D77 and D80. *Less informative:* copy or move, and a block's captures. *Checks:* the rest, each establishing (b), establishing (c), or protecting only runtime safety (the requirement that borrows passed to a call that closes off be whole). "A returned borrow must be whole" is membership, not only runtime safety (test 2).
- *The "same" kind rests on five lemmas, not one per site:* membership; the soundness of `valType?`; footprint coverage; conversion soundness, which must be monotone over `⊥` ([Conv-bot], (V9)); and well-formedness (Section 4.3.4).
- *Item 1, settled by a probe: a simplification of the proof, not a gap.* When a value is checked whole while part of it is lent, the rule checks the lent part as an unknown of the borrow's *declared* type, since the borrow may still write anything of that type there. The checker instead uses the type read off the lent part's current content (`lentUnknown`). The audit worried about the case where that type cannot be read. A probe found that this case rejects rather than accepts: the loan stays in the value, and [Repack]'s check reads the same unreadable type and reports that the part is not of its type. The case is also effectively unreachable, since a lent index field is first-order data, whose content's type can always be read (a constructor value records its parameters, an unknown has a stored type, a sealed program has its head's codomain); four attacks and an attempted closed proof of `False` were all rejected by [Repack]. Reading the borrow's declared type, as the rule says, would change no verdict. It would only remove this check's dependence on the soundness of `valType?`, one of the five lemmas behind "same".

- *Item 2, open: the class of an embedded call head.* When a call's head is a value with no written type, such as a sealed function value, its class is read off the value's Π-type. That agrees with the declared position only because Π-types of different class are not convertible (D54), and no test exercises that check. The fix: a witness test, or carrying the class with the value, as D80 does for captures.
- *Untested invariant checks:* that a closure in a block arm captures nothing the block borrows, in the copy-type case, and that a recursive function occurs in its body only as a call's head. Each needs a test that it rejects something.

6.4 Test 2: a higher-order slice

What it checks. The route on a slice with the features that the first-order fragment F leaves out (F is the part of the calculus for which the Ochr paper proves soundness: no closures, no function parameters, no type families): function parameters ranging over every set function, a closure built in a run and called later, [Conv-fun] (functions compared by their generic observations), and a function taking a function that returns a borrow. It checks three cases that are delicate for any proof about this machine (an early end chosen by a branch, as in `Pick`; early ends with pairs after a [Drop]; and resolution commuting with refinement through inert loans), and the census's reading of `⊥` on higher-order code.

Why not F. F is first-order: no closures, no function parameters, no type families. Redoing its proof this way is at most a warm-up, and its outcome says little about the whole calculus.

Result: states have a meaning, and the step lemmas hold on the slice (`ochr/docs/27`, 6 November 2026; a paper result, with the meaning of states checked in Lean on a toy symbolic machine in `ochr/core/lean/Ochr/Model/State.lean` and `State2.lean`, which build with no `sorry` and use only the axioms `propext` and `Quot.sound`). With it, all four planned tests have passed; the one failure this test found is a fifth closed proof of `False`, conversion over `⊥`, since fixed on main by D83.

- *A state means a function of a future.* A future gives each live borrow its final content. A place that lent its content means its final content; a place holding the borrow means its current content, which is what a read through it sees; a sealed fill means its program, with its hole read from the future; `⊥` means no information. Ending a borrow fixes the future at the holder's current content and forgets the holder, so it loses only information, and that is what the checker's [End] does by substitution. The Lean check found this shape necessary: a single function from final contents to resolved contents cannot express a read through a borrow.
- *`Id`'s meaning.* Each observation ends every live borrow of its copy, so it depends on the assignment alone, and `Id` compares the results and every position neither side moves. The checker's conjunction over the footprint's owners is then exact, given footprint coverage.
- *Both obligations test 0 left hold, with no new rule.* No stored type holds a live loan: types are formed on private copies after [Access] has ended the loans in what they read, and the one type computed over a lent field, an open dependent value's, quantifies over the hole, as D64 amended and D73 require. Every accepted function returning a borrow has an injective backward function, by an invariant that every hole is placed injectively in the state. [Access], D67 (an assignment ends the loans it overwrites), D65 (a borrow of a dying local cannot escape) and (C8) (arguments hole-free at [Close]) keep it, and every called function is good because types mean only good elements.
- *The three delicate cases are routine.* `Pick`'s early end is a definition (a fill is a function of its hole) and an end that loses only information. Early ends with pairs after a [Drop] are an end, which is below whatever the shape of the ended borrow's content, since reborrows travel with it and are read from the same future. And resolution commuting with refinement through inert loans is meaning preservation for a fill's re-run, with the hole as one more unknown.
- *Monotonicity.* `Eq`, `J`, matches and [Close]'s rows are monotone over `⊥`. Conversion was not: its identity test identified two values that hold `⊥`, which is the fifth closed proof of `False` (Section 9). The fix, [Conv-bot] with its counterpart for [Rewrite], is on main since D83, and changed no existing verdict.
- *The form of the main theorem.* The checker's run is below a *reference run* of the term on meanings (Definition 13): the denotation as a state transformer, which applies the meanings of functions at calls instead of

running their bodies, so it is not a second machine run. Each step is exact, an end the reference does not make, or an over-approximation. Theorem 14 states it this way.

- *The higher-order slice.* A function parameter means any good function, non-computable ones included, and a call through it closes off exactly. A closure built in a run is fixed when it is formed, since its captures hold no loan, and a later call applies its meaning. [Conv-fun] is sound by function extensionality on good functions. A function taking a function that returns a borrow, `UseG (g : Π(x : &Nat). &Nat) (a : Nat) : Nat := (let r = g(&a); *r := 5; clone(a))`, means $(G, n) \mapsto \mathrm{back}_G(n)(5)$, so two different writes give two different results, by injectivity.
- *In Lean.* [Access] is derived the checker's way; ending one borrow reaches every reborrow, and shallow ends are sound, so early ends with pairs need nothing more; nested holes hold; and [Drop] loses only information. (WF5), loan-free arguments at a call, turns out to be about causality: with a live loan in an argument, a read now would depend on a write made later, so the meaning of a state needs it, not only runtime safety. Lean's own gaps: the step lemmas at field paths are checked only on the runs it encodes, and the preservation of (WF2) is assumed.
- *A correction to the census.* "A returned borrow must be whole" is membership, not only runtime safety: a returned borrow holding $\bot$ has a forward value with no meaning. The requirement that borrows passed to a call that closes off be whole stays on the runtime side.

What it leaves. The lemmas this test assumed or argued without checking: footprint coverage; the soundness of `valType?`; the preservation of well-formedness, where (WF2) needs (WF3); [Open] and [Repack] beyond the lent-index case; and erased runs inside the reference run. These are the residuals of Section 8.

6.5 Test 3: type safety on the higher-order slice

What it checks. Target B on test 2′s higher-order slice: error reflection, runtime `J`, erasure and termination, for closed runs of accepted data functions.

What would fail it. A closed run of an accepted function that goes wrong where the reference run does not, or whose result differs from the reference run's.

Result: type safety holds on the slice, up to two open lemmas, and no checker bug was found (`ochr/docs/29-type-safety-slice.typ`; checked in part in Lean in `Ochr/Model/Safety.lean`, a concrete runtime on test 2′s toy machine, which builds with no `sorry` and uses only the axioms `propext` and `Quot.sound`). It is stated against D84′s drop-at-the-join semantics, not yet merged, under which a moved place means "moved on every path" and moves are exact between the generic call, a closed run and the reference run.

- *The target, sharpened.* Function arguments must be built by accepted code, since a closure from outside the program could loop; proof parameters must be true in the model, which also makes runtime `J` reachable at non-convertible endpoints. Target B is stated so.
- *Error reflection.* Of the machine's 140 error sites, value errors (11) reflect to the reference run, since at a closed state a value means itself; borrow errors (18) reflect to it too, operation by operation in Lean, using the injectivity of backward functions, except at one-element types, where a loan can sit that no meaning depends on (`UseDirect`); there they reflect to the checker's derivation, by a lemma about borrow structure that is sketched. Static and typing errors (93) are unreachable at runtime, and internal assertions (18) are invariants. Following * from a non-borrow needs typing in the invariant.
- *Runtime `J`.* The checker's rule gets stuck at runtime on two encodings of one function ($n + 0$ against $0 + n$ in Lean); returning the transported term is sound by membership and reads nothing (Lean: `castRT_safe`). Decided as D85.
- *Erasure* holds by construction, with runtime `J` its one extra clause. *Termination* holds for recursion by name; calls through function values need a predicative realizability relation, not yet built. *Well-formedness* at runtime is partly proved in Lean.
- *Probes.* The likeliest failure of the borrow-structure lemma, η at `Unit` in a closed-off call's fills, does not occur: two programs aimed at it are rejected, as predicted.

What it leaves. The open lemmas listed in Section 4.3.6.

6.6 Audit: what a context can observe

What it checks. For Target C: every channel through which a context can observe a term (its result, the places it writes or moves, where a borrow it assigns points, and any other), compared with `Id`'s footprint, on main and under D84.

What would fail it. A channel `Id` does not observe, as `RePoint` is for where a borrow points.

Result. Pending (`ochr/docs/30-id-adequacy-audit.md`).

7 The lemmas and their status

Figure 1 draws what the plan's lemmas depend on, and Figure 2 lists their statuses.

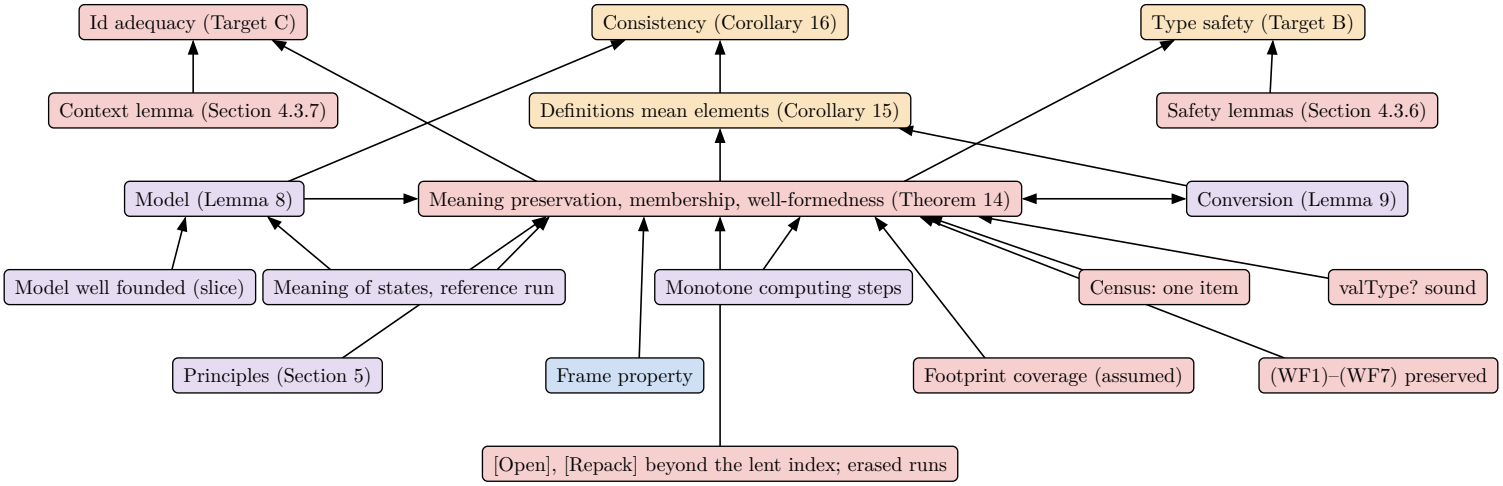


Figure 1: The plan's lemmas and their dependencies; an arrow points from a lemma to one that uses it, and the double arrow marks the induction conversion soundness shares with the main theorem. Colours: proved on the heap slice, reduced, paper argument, open, open design question.

Lemma	Status	Depends on; notes
Consistency (Corollary 16)	**reduced to Corollary 15, the model**	Lean statement `Ochr.Consistency.consistency` (`sorry`)
Definitions mean elements (Corollary 15)	**reduced to Theorem 14, Lemma 9**	
Meaning preservation, membership, well-formedness (Theorem 14)	**open**	reference-run form; the step lemmas hold on a higher-order slice (test 2), much of it in Lean; test 1 found every fixed proof of `False` to be a step where one part fails; residuals in Section 8
Model (Lemma 8)	**paper argument** most of a slice in Lean	conditions (I1)–(I9), all enforced by the checker; well founded on a slice by test 0, beyond it Q2; `Ochr/Model/Slice.lean`, no `sorry`
Meaning of a state, reference run (Definition 11, Definition 13)	**paper argument** a toy machine in Lean	futures (test 2); `Ochr/Model/State.lean`, `State2.lean`, no `sorry`; ⊥ as no information
Types hole-free; backward functions injective	**paper argument**	test 2; from [Access], D64, D65, D67, D73 and (C8)
[Open], [Repack] beyond the lent index	**open**	covered by test 2 only for a lent index field; Q3
Erased runs in the reference run	**paper argument**	argued in test 2, not checked; Q2
Conversion sound (Lemma 9)	**paper argument**	(V1)–(V9), argued under assignments; shares Theorem 14′s induction; (V9) is [Conv-bot] (D83); known incompleteness `SelfBot`
Monotone computing steps	**paper argument**	`Eq`, `J`, matches and [Close]'s rows are monotone over ⊥ (test 2); conversion and [Rewrite]'s occurrence search since D83 ([Conv-bot]), which closed the fifth proof of `False`
Decisions sorted (census)	**open**	one item, an embedded call head's class; reading the lent-part check's type from the borrow's declaration would only simplify the proof
`valType?` sound	**open**	one lemma about one function; D82 (on main) removed its uses at bindings
Footprint coverage	**open**	assumed by test 2; the extra owners at a call site by the injectivity of contexts and (O4)
Frame property	**proved on the heap slice**	a lemma both targets use, not a target: a call's meaning changes only its result and borrowed places;
	reduced to (L1)–(L7) on checker	`Ochr.Heap.frame`, `call_effect` (heap slice)
Type safety (Target B)	**reduced to the reference run; (a)–(e) of Section 4.3.6**	test 3 passed on the slice, no checker bug; open lemmas in Section 4.3.6
`Id` adequacy (Target C)	**reduced to Theorem 14, Target B, the context lemma**	a draft statement; neither other target implies it
Context lemma	**open**	what a context can observe of a term is what `Id` compares; the audit of Section 6.6 pending; known gaps: moves (D84, not merged) and `RePoint` (observe assigned borrow variables)
Error reflection	**paper argument** Lean per operation	value and most borrow errors to the reference run; at one-element types, borrow errors to the derivation; not yet over a whole run
Borrow-structure lemma	**paper argument**, sketched	the derivation is at least as strict as the closed run on borrow structure; D84 makes moves exact
Realizability	**open**	termination through function values; predicative (Q4)
Typing in the runtime invariant	**open**	for following * from a non-borrow
Runtime `J`	**paper argument**	decided: the erased runtime's `J` is its transported term (D85, branch `runtime-j`, not merged); sound by membership (`castRT_safe`); the checker keeps its rule
Erasure	**paper argument**	by construction; runtime `J` its one extra clause
Termination	**reduced to [Rec]; realizability**	recursion by name: the guard never fires; through function values: realizability, open; Ochr stays total
Well-formedness, (WF1)–(WF7)	**open**	(WF2) needs (WF3), assumed by test 2 and in Lean; (WF5) is about causality; two of its checks have no test

Figure 2: Status of every lemma the plan uses.

8 The order of work

1. **The residual lemmas**, which test 2 assumed or argued without checking: footprint coverage; the soundness of `valType?` (D82, on main, removed its uses at bindings); the preservation of well-formedness, where (WF2) needs (WF3); [Open] and [Repack] beyond the lent-index case (Q3); and erased runs inside the reference run (Q2).
2. **The census's open item**: a witness test for the class check on Π-types, and a rule change if it fails; tests for the two invariant checks that have none. Optionally, the lent-part check can read the borrow's declared type, which changes no verdict and removes one use of `valType?`.
3. **Type safety**: landing D85, runtime `J` (branch `runtime-j`); and the open lemmas test 3 left (Section 4.3.6): the borrow-structure lemma, realizability for calls through function values (Q4), typing in the runtime invariant, error reflection over a whole run, and well-formedness at field paths with (WF1), shared with consistency.
4. **`Id` adequacy**: the audit of what a context can observe (Section 6.6); the context lemma; observing each assigned borrow variable by writing a fresh σ_w through it, which closes `RePoint`; and D84 for moves (Section 4.3.7).
5. **The model and the meaning of states, on paper, for the whole calculus** (Q2): what the slices covered only on paper or not at all (user inductives and dependent constructors in Lean, inductives declared `abstract` or `implemented by`, `split f in t`, `rewrite`, records), with conversion soundness (Lemma 8, Definition 11, Definition 13, Lemma 9).
6. **The step lemmas, on paper**, one per checker rule, against the reference run, each stating which of (a), (b) and (c) the step must keep and which it establishes, with the five lemmas behind "same" (Section 4.3.4). Large but routine; the standing preference is that size costs nothing and risk is what counts.
7. **The Lean statements** of the three targets (Q1 for consistency), and mechanisation, with meaning and membership built together, as both Lean encodings already do.

Not planned: a heap port. The checker's value layer is not to be ported to the heap design, and the branches `heap-close` and `port-step1` are not to be merged. The heap slice stays as a Lean artefact, cited for the frame property.

9 Where the plan could fail

The risks below are places where a lemma might be *false*, not merely unproved.

`Id` adequacy needs rule changes (Section 4.3.7). `RePoint` shows that `Id`'s footprint misses where an assigned borrow variable points, moves by one side need D84, and the audit of Section 6.6 may find further channels. These are gaps in substitution, not proofs of `False`.

Type safety's open lemmas may fail (Section 4.3.6). Error reflection fails as a lemma at one-element types (`UseDirect`), and the borrow-structure lemma that covers them is only sketched; the realizability relation for calls through function values is not built; typing must be shown to be in the runtime invariant; and runtime `J` rests on D85, not yet merged. Test 3 is stated against D84′s drop-at-the-join semantics, which is not merged.

The residual lemmas may be false. Test 2 assumed footprint coverage, the soundness of `valType?` and the preservation of well-formedness, and covered [Open] and [Repack] only for a lent index field and erased runs only by argument (Section 6.4). Each is a place a lemma could fail; [Repack] is where one of the October proofs of `False` was. On the slices, the model (test 0) and the meaning of states (test 2) held.

A decision's class may come from a value (the census's second item). The class of an embedded call head is read off its Π-type, and agrees with the declared position only through a check on Π-types that no test exercises.

The well-formedness invariant may not be preserved. In particular, an early end of a borrow must not leave a reborrow alive and readable. Two of the invariant's checks have no test.

Facts that hold only at reachable states. Four natural statements about the checker's decisions are false at arbitrary states and true at the states checked programs reach (found by stating them in Lean; none is a checker bug):

- the class and [Close] row of a function type commute with refinement only for Π-type values and only from the generic to the instance (an unknown A : `Type` refined to a Π-type reads no class generically);
- the [Close] row of a stuck block does not commute (an unknown match type refined to `Unit` flips it from data to `Unit`, harmlessly);
- footprint containment fails for a refinement that makes an unknown type a borrow type (excluded by requiring borrow-free refinements, which is what refinements are);
- declared readings survive refinement only at states of checked programs (a sealed call of an unchecked global can re-run to a type).

So a step lemma may not assume a property of arbitrary states, which is why membership, part (b), is proved inside the induction.

Closed proofs of `False` found while the principles were being derived. On the checker of October 2026, three accepted programs proved `False`; a referee found a fourth on 2 November; and a fifth was found on 6 November, while test 2 was being written. All five are fixed and are now rejected tests:

- writing through a returned borrow of an index field, so that a value's length and its dependent field disagreed while the generic path trusted the value as whole (fixed: a part still lent at a whole-again point is an unknown of its type);
- a stuck `J` closed off without checking its borrowed places at the end of its transported term (fixed: a `J` block is checked at its return, like any block);
- a recursive local function capturing a value it splits on: the split rewrote the capture, the recursion check identified calls by comparing function values, and missed them (fixed: recursive calls are recognised by name);
- `EtaBoom` (Section 4.3.8): leaving a split arm kept the η-refinement of a one-constructor unknown, whose fields' types had been read in the arm's world, so a sibling arm reused a field typed `Void` where a `Nat` was due (fixed: an η-refinement, like a generalisation record, lives until the arm that made it ends);
- `Same` (6 November): at an unknown `n`, each side of an `Id` is a stuck block `match n { Z => (let y = x; ()), S _ => (x := c; ()) }`, which takes `x` by move because its `Z` arm moves it; every observation then leaves `x` as ⊥, so a hypothesis (with `c = 5` on both sides) and a goal (with `5` and `6`) both compute to the stuck `Eq(Nat, ⊥, ⊥)`, and conversion's identity test identified them; at $n = 1$ they are $5 = 5$ and $5 = 6$. The fix's own audit found two more routes to `False` through the same identification, so this is one bug with three routes, all identifying values that have lost information: conversion's identity test (`Same`); `trans`, which composed $5 =$ ⊥ and ⊥ $= 6$ into $5 = 6$ because conversion at the middle point treated the two ⊥ as equal (`TrBotBoom`); and `rewrite`, whose occurrence search uses equality of representations, so `rewrite ← h` with `h : Eq(Nat, ⊥, 5)` put `5` in place of another observation's ⊥ (`RwBotBoom`), a separate code path from conversion. Fixed on main by D83 (`98e711ed`): [Conv-bot], a value containing ⊥ is convertible with nothing, itself included, and [Rewrite] finds no occurrence of a value holding ⊥.

Test 1 (Section 6.2) found each to be a step at which one part of Theorem 14 fails: (b) for the first three (the third through the guard of a recursive function), (a) for the fourth. The fifth, found after test 1, fits none of the three parts: it is a failure of the monotonicity Theorem 14 requires of every step that computes on a state (Section 4.3.4), by three routes that each identify two values holding ⊥, which may stand for different values. Since [Rewrite]'s occurrence search is a separate code path from conversion, monotonicity has to cover both. The principles of Section 5 name the obligations and single points in the checker enforce them, and probes of five further areas found nothing; but a list of principles can be incomplete, which is what the plan exists to rule out. Independent evidence: a route that translates each accepted program into Lean and has Lean's kernel check it refused, for the second and third programs, the certificates that a checker still accepting them produced, each for the actual reason the program was wrong.

Incompletenesses are not risks, but they shape the theorem. Several true programs are rejected because the generic call is stricter than an instance: two copies of a function that inspects its argument are not convertible; a match on a generalised call is not convertible with the call; a block's capture over-approximation rejects a closure inside a statement inside an arm, and a block that takes a place by `&` because an erased term assigns it; changing an index while a dependent field is lent is rejected even when the field's type is the same at every

index; large elimination from a Forded proposition is refused at an index where only one constructor is possible; an impossible arm is refused at an unknown index; and, since D83, a type holding `⊥` is not convertible with itself (`SelfBot`). Each confirms that some steps over-approximate, so Theorem 14 must allow a less informative result (Definition 12), not only an equal one.

Universes in Lean (Q1). The uniform Lean statement may be unprovable in Lean.

Rules and checker. The Lean statement is about the checker; its correspondence with the rules is tested, not proved. A proof against the checker makes this moot; a paper proof about the rules does not.

10 Open design questions

Open design question Q1 (universes in Lean). The paper model fixes one inaccessible per universe level a program uses, and test 0′s Lean encoding takes the bounded option (levels 0 and 1). Which statement should the Lean theorem make over all programs: one inaccessible per level for all levels at once (unprovable in Lean by Gödel if Ochr is as strong as Lean, which is unknown: Ochr has impredicative `Prop` and every $\texttt{Type}_i$, but no W-types, so the inaccessibles may even be unnecessary), bounded levels, a statement relative to an assumed model, or one statement per program?

Open design question Q2 (the model and the reference run beyond the slices). The model's definition (Section 4.1) and the reference run (Definition 13) are checked on slices. For the whole calculus: what clause gives a meaning to inductives declared `abstract` or `implemented by`; do the slices' constructions carry over, in Lean, to user inductives beyond `Pair` and `Unit`, dependent constructors, type functions of any signature and `Eq` beyond `Nat`; and do erased runs inside the reference run, `split f in t`, `rewrite` and records, covered so far only by argument or citation, keep meaning?

Open design question Q3 (open dependent values beyond the lent index). [Open] and [Repack] are argued only for the case where a lent field is an index field (Section 6.4). Does [Repack] establish membership, under every future of the holes still lent, for every way a dependent value can be opened: writes, borrows and moves through any field, nested dependent constructors, and values opened in one call and repacked in another?

Open design question Q4 (realizability for calls through function values). Type safety needs calls through function values to terminate, which [Rec]'s measure does not cover (Section 4.3.6, (d)); consistency needs no termination. The candidate is a realizability relation between runtime values and meanings, in the style of Tait and Girard, defined by the model's recursion, at a type variable over pairs of a set and a relation, and predicative since proofs are erased. Can it be defined so, and does every accepted function realize its type?

11 An earlier version

An earlier version of this plan, which followed a different route, is preserved on the git branch `checkpoint/before-meaning-route` (`da8b260b`). The referee reports summarised in Section 4.3.8 were written against it and are in `ochr/docs/reviews/`.